

Initiation à la méthodologie DevOps, Gitlab CI/CD et conteneurisation Docker

Partie 4 : Kubernetes, le cluster qui tient l'état voulu

1 - Révisions rapides

Hier, la Todo API a fini la journée en production, sur une machine qu'on connaît par son nom, `vm-prod`. Trois choses tiennent déjà debout :

- **le chemin complet** : un `git push` sur `main` suffit, la pipeline construit l'image, se connecte en SSH à `vm-prod` et y installe la nouvelle version, sans qu'aucune commande soit tapée à la main,
- **la vérité avant le départ** : la même pipeline teste l'API contre une vraie base PostgreSQL, et l'application expose `/health` et `/metrics`, que Prometheus scrape en continu pendant qu'un tableau de bord Grafana répond en un coup d'œil aux quatre questions qui comptent,
- **le papier pour la nuit** : une procédure de déploiement versionnée dans le dépôt dit quoi taper en cas de panne, avec son retour arrière, et elle a déjà servi pour de vrai, un camarade s'en est servi hier pour réparer une machine qu'il n'avait jamais touchée.

Trois limites nommées hier n'ont pas bougé depuis, et c'est très exactement elles qui ouvrent la journée d'aujourd'hui :

- chaque déploiement coupe le service, quelques secondes, le temps qu'un conteneur meure et qu'un autre prenne sa place,
- si l'application plante toute seule à 3h du matin, rien ne la relève. Le script d'hier ne s'exécute qu'au moment d'un push, jamais de lui-même,
- et si le trafic décuple d'un coup, `docker run` ne sait grossir qu'à la main, un `stop` puis un `run` avec un conteneur de plus, pendant qu'un humain regarde un graphique et décide.

J3, on livrait sur une machine, et c'est nous qui réparions. J4, on décrit l'état voulu à un cluster : lui déploie sans coupure et répare ce qu'il sait réparer. Notre travail devient de reconnaître ce qu'il ne sait pas réparer.

Planning de la Journée

1. **Décrire l'état voulu plutôt que dicter les gestes** : le modèle déclaratif contre le modèle impératif, et la boucle qui compare la réalité à ce qu'on a écrit, en continu, sans qu'on ait à la relancer.
2. **Ouvrir le capot d'un cluster** : le control plane qui décide, les nodes qui exécutent, le trajet complet d'un `kubectl apply` jusqu'au conteneur qui tourne pour de vrai, et ce que k3s puis k3d simplifient sans rien trahir du vrai Kubernetes.

3. **Le vocabulaire qu'on va vraiment manipuler** : Pod, Deployment, Service, ConfigMap, Secret, PersistentVolumeClaim, Ingress, Namespace, et les commandes `kubectl` qui servent tous les jours.
4. **Mettre à jour sans couper, et encaisser la charge** : le rolling update compté pod par pod, les sondes readiness et liveness, les replicas et les limites de ressources.
5. **Lire un cluster qui va mal** : `describe`, les events, les états d'un pod qui refuse de tourner, et brancher le Prometheus d'hier sur ce nouveau terrain.
6. **La procédure de déploiement, réécrite pour une cible qui n'est plus une machine** : ce qui change, ligne par ligne, dans le document écrit hier.
7. **À votre tour : du serveur unique au cluster** (*après-midi*).

2 - Décrire l'état voulu au lieu de dicter les gestes

Une machine tourne, seule, la nuit. Le conteneur qui portait l'application vient de mourir, une fuite mémoire, une dépendance externe qui a mal répondu, peu importe la cause exacte. Personne n'est devant un écran. Le script de déploiement d'hier ne se déclenche qu'à un `git push`, et cette nuit-là, personne ne pousse rien. Au matin, l'application est toujours down, exactement comme elle l'était à 3h.

Qu'est-ce qui aurait dû se passer, entre le crash et le matin ?

L'essentiel du modèle déclaratif

Deux façons de dire à une machine ce qu'on veut

Jusqu'ici, tout ce qu'on a écrit était **impératif** : une suite de gestes précis, dans un ordre précis, exécutés une fois. `docker stop`, puis `docker rm`, puis `docker run`, chacun déclenché par un humain ou par une pipeline qui répond à un événement. Une fois la séquence terminée, plus rien ne surveille ce qui se passe ensuite. Si le conteneur meurt cinq minutes après le dernier `docker run`, la séquence ne le sait pas, elle est déjà terminée.

```
# Le script de déploiement d'hier, exactement tel qu'on l'a écrit :  
# trois gestes, dans l'ordre, exécutés une seule fois au moment du push  
docker stop hello-app || true  
docker rm hello-app || true  
docker run -d --name hello-app -p 8080:80 monpseudo/hello-app:$SHA
```

Rejouer cette séquence répare bien le conteneur mort, mais seulement si quelqu'un, ou quelque chose, la relance. Rien dans ces trois lignes ne dit "et recommence si ça retombe" : la garantie s'arrête à la ligne `docker run`.

Modèle impératif ? Une suite d'instructions qui décrit *comment* faire, étape par étape. Elle s'exécute une fois, produit un résultat, et s'arrête. Rien ne garantit que ce résultat tienne dans le temps.

Le **modèle déclaratif** renverse la question. Au lieu de décrire les gestes à faire, on écrit l'état qu'on veut voir exister, en continu : "je veux que trois copies de cette application tournent, à tout instant, quoi qu'il arrive". Ce texte, on ne l'exécute pas une fois, on le confie à un système qui le garde sous les yeux en permanence, et qui agit de lui-même chaque fois que la réalité s'écarte de ce texte.

Le modèle déclaratif, en une ligne : une description de *ce qu'on veut*, pas de la façon d'y arriver. Un système tiers compare cette description à ce qui existe réellement, et corrige l'écart, tout seul, sans qu'on le lui redemande.

L'image qui fait comprendre d'un coup, c'est le thermostat. On ne dit pas au chauffage "allume-toi maintenant, puis éteins-toi dans vingt minutes". On règle une température, 20°C, et l'appareil chauffe ou coupe, tout seul, aussi longtemps que nécessaire, pour que la pièce reste à cette température. Personne ne relance le thermostat chaque fois qu'une fenêtre s'ouvre et fait chuter la chaleur : il l'a déjà remarqué, et il corrige.

Kubernetes applique exactement ce principe à des conteneurs. Un texte dit "je veux trois copies de cette application, toujours", et un système travaille en boucle, jour et nuit, pour que ce texte reste vrai. La nuit du début de section, avec un vrai cluster à la place d'un `docker run` isolé, le conteneur mort aurait été remplacé avant même qu'un humain ait eu le temps de dormir dessus.

Le passage à l'échelle rend la différence encore plus nette. Faire passer une application de 3 à 10 copies, en impératif, veut dire écrire et exécuter 7 `docker run` de plus, sur 7 emplacements qu'il faut d'abord trouver. En déclaratif, un seul chiffre change dans le texte, `replicas: 10`, et le système se charge du reste : où poser les 7 nouvelles copies, dans quel ordre, avec quelles vérifications. La complexité ne disparaît pas, elle change de camp : elle quitte la personne qui écrit le texte pour aller vers le système qui l'exécute.

La boucle en trois gestes : observer, comparer, agir

Ce système qui garde le texte sous les yeux ne fait, en réalité, que trois choses, encore et encore :

- il **observe** ce qui tourne réellement, à cet instant précis,
- il **compare** cet état réel à l'état écrit dans le texte,
- s'il y a un écart, il **agit** pour le refermer, et rien de plus s'il n'y en a pas.

Prenons un texte tout simple, écrit sur un exemple neutre, une application "hello" qui n'a rien à voir avec la Todo API : "je veux 3 copies de hello qui tournent". Voilà ce que fait la boucle, minute après minute, sans qu'on la relance jamais :

Moment	Ce qui tourne réellement	Ce que dit le texte	Action
Démarrage	0 copie	3 copies	On en crée 3
Une minute plus tard	3 copies	3 copies	Rien, l'écart est nul
Une copie plante	2 copies	3 copies	On en recrée 1
Quelqu'un supprime une copie à la main	2 copies	3 copies	On en recrée 1, encore
Une minute plus tard	3 copies	3 copies	Rien

Rien de magique là-dedans : c'est une comparaison, répétée, avec une action minimale à chaque fois qu'elle révèle un écart. C'est précisément ce mécanisme qui aurait remplacé le conteneur mort de 3h du matin sans qu'on ait besoin d'écrire un seul `if crash then restart`.

On a croisé Kubernetes en une ligne au Jour 2 : un système qui maintient l'état déclaré, quoi qu'il arrive. Aujourd'hui, on ouvre le capot, on regarde comment ce système est fait, et on apprend à lui parler pour de vrai.

Creusons : la boucle de réconciliation

Cette boucle observer-comparer-agir a un nom, dans le vocabulaire du projet : la **boucle de réconciliation**.

La boucle de réconciliation, pour la nommer d'un coup : le mécanisme qui tourne sans interruption dans un cluster Kubernetes, compare l'état déclaré à l'état réel, et corrige l'écart. Elle ne s'arrête jamais, elle ne se relance jamais, elle boucle.

Un détail change tout, et il mérite qu'on s'y arrête : cette boucle ne s'exécute pas une fois au moment du `kubectl apply`, puis s'arrête. Elle continue de tourner bien après, indéfiniment, tant que le cluster existe. C'est ce qui distingue un `docker run` d'un état déclaré : le premier produit un résultat à un instant T, le second reste vrai à tout instant après T, parce que quelque chose veille dessus en continu.

Que se passe-t-il quand l'écart ne peut pas se refermer ? La question mérite d'être posée maintenant, elle revient plus tard dans la journée. Trois copies demandées, mais aucune machine du cluster n'a la mémoire disponible pour en accueillir une troisième, par exemple. La boucle continue de comparer, elle continue de vouloir agir, mais l'action échoue à chaque tentative. Le pod reste bloqué dans un état d'attente, visible, et c'est justement ce genre de blocage qu'on apprendra à lire en section 6. La boucle ne ment jamais sur ce qu'elle n'arrive pas à faire, encore faut-il savoir où elle le dit.

Un deuxième cas mérite d'être posé, plus sournois que le premier, parce qu'il ne casse rien tout de suite : *que se passe-t-il si quelqu'un modifie le nombre de copies à la main, avec `kubectl scale`, sans toucher au fichier YAML sur disque ?* Rien ne s'y oppose sur le moment, la commande fonctionne, le cluster obéit. Mais le fichier, lui, n'a pas bougé : il continue de dire 3, pendant que le cluster tourne à 5. Le prochain `kubectl apply -f` du même fichier ramène tout à 3, sans prévenir, en écrasant purement et simplement la modification faite à la main. Ce petit écart entre "ce que dit le fichier versionné" et "ce que le cluster fait réellement" porte un nom en entreprise, le **drift**, et la meilleure protection contre lui reste la plus simple : ne jamais modifier un cluster autrement que par le fichier versionné, pour que ce fichier reste, à tout instant, une photographie fidèle de ce qui tourne. Poussée à son terme, cette discipline porte un nom, le [GitOps](#) : un dépôt Git comme unique source de vérité pour l'état d'un cluster, où un outil applique automatiquement tout changement mergé, et où plus personne ne touche `kubectl apply` à la main. Retenez le nom, l'idée elle-même n'a rien de neuf : c'est exactement le principe déclaratif de cette section, poussé jusqu'au bout de la chaîne.

Retenez le principe, il revient partout aujourd'hui : un `Deployment` qui maintient un nombre de copies, une mise à jour qui remplace des pods un par un sans jamais couper le service, un pod supprimé à la main qui réapparaît tout seul. Ce sont quatre visages différents de la même boucle, celle qu'on vient de voir tourner sur trois copies de "hello".

3 - Ce qu'il y a dans un cluster

Une commande, `kubectl apply -f deployment.yaml`, et quelques secondes plus tard, un conteneur tourne pour de vrai, quelque part. Entre les deux, plusieurs pièces se sont parlées, dans un ordre précis, et aucune d'elles ne s'appelle Docker directement.

Qu'est-ce qui se passe, exactement, entre la commande tapée et le conteneur qui répond ?

kubectl, en une ligne : le client en ligne de commande qui pilote un cluster Kubernetes, celui qu'on vient de voir. Toute interaction avec un cluster passe par lui. Doc officielle : [kubectl](#).

Sur votre laptop, jusqu'ici, une seule machine jouait tous les rôles à la fois : elle décidait de lancer le conteneur, et elle le faisait tourner, sans distinguer les deux métiers. Un cluster sépare ces deux rôles sur des machines différentes, et c'est précisément cette séparation qui permet à l'ensemble de survivre à la panne d'une seule d'entre elles.

L'essentiel de l'architecture

Un cluster Kubernetes se découpe en deux familles de machines, qui ne font pas le même métier. La première décide, la seconde exécute. Aucune des deux ne sait faire le travail de l'autre.

Ces noms, etcd, API server, scheduler, controller manager, étaient juste cités entre parenthèses au Jour 2, pour dire qu'un vrai Kubernetes est lourd à installer. Chacun devient maintenant une pièce qu'on sait situer, et dont on connaît le métier.

Le control plane : celui qui décide

Le control plane, en clair : l'ensemble des composants qui prennent les décisions du cluster, surveillent son état et le réparent. Il ne fait jamais tourner une seule ligne de votre application, il commande. Doc officielle : [Kubernetes Components](#).

Quatre pièces composent le control plane, chacune avec un métier précis :

- **l'api-server** : la porte d'entrée unique du cluster. Absolument tout passe par lui, `kubectl`, un script de pipeline, un autre composant interne. C'est lui qui reçoit un `kubectl apply` et qui décide s'il est valide avant même d'y toucher.
- **etcd** : la mémoire du cluster. Un [etcd](#) est une base de données clé-valeur distribuée qui stocke l'état voulu et l'état réel de tout ce qui existe dans le cluster, quels objets, combien de copies, où. Si etcd est perdu, le cluster oublie tout ce qu'il savait : plus aucun objet, plus aucune trace des Secrets ou des ConfigMaps, un cluster vide qui ne sait même plus qu'il a existé. C'est la seule pièce du control plane dont la perte n'a pas de remède rapide, ce qui explique pourquoi, en production, sa sauvegarde régulière est traitée avec le même sérieux qu'une sauvegarde de base de données.
- **le scheduler** : le placeur. Quand une nouvelle copie doit démarrer, c'est lui qui choisit sur quelle machine la poser, selon la place disponible et les contraintes déclarées.
- **le controller manager** : le réparateur en boucle, celui qui fait tourner la boucle de réconciliation vue en section 2. Il compare en permanence ce qui est voulu à ce qui tourne, et corrige l'écart.

Une réserve de vocabulaire, glissée en passant : sur un cluster hébergé chez un fournisseur cloud, un cinquième composant existe, le [cloud controller manager](#), qui parle au fournisseur pour créer un répartiteur de charge ou attacher un disque. Sur les machines qu'on administre soi-même, comme celles d'aujourd'hui, il n'a rien à faire.

Les nodes : ceux qui exécutent

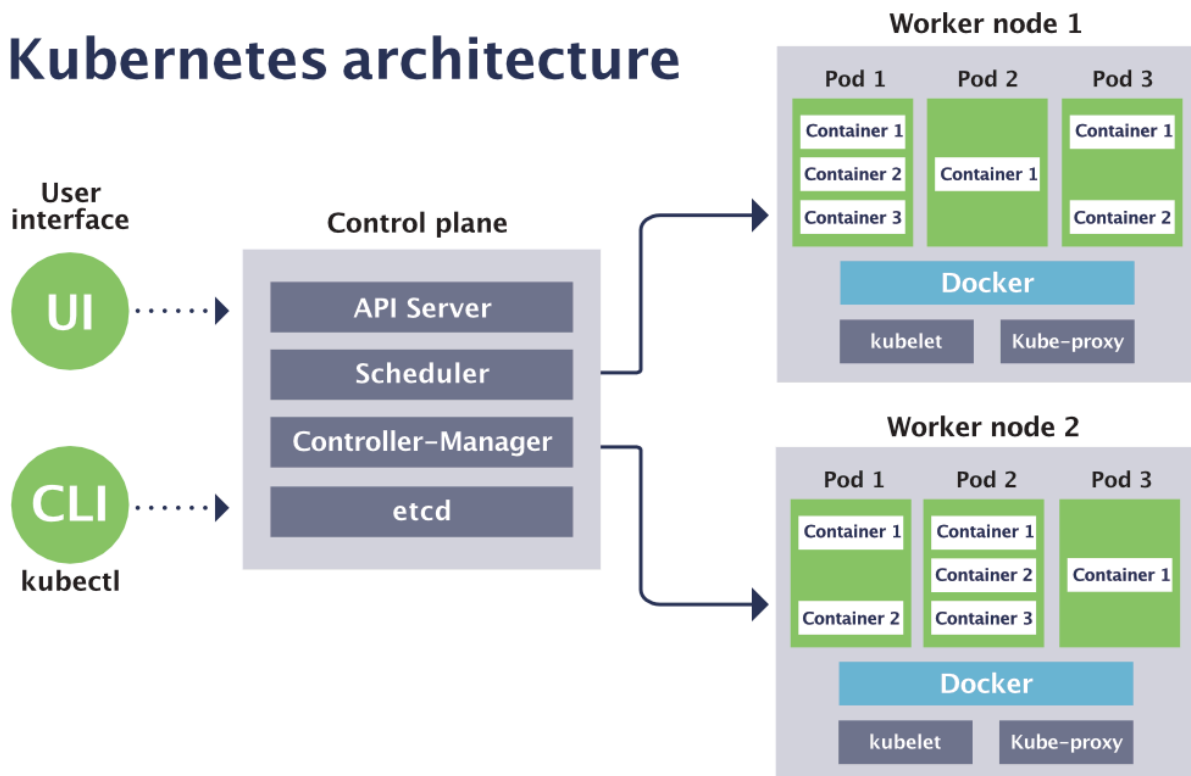
Un node, c'est quoi ? Une machine, physique ou virtuelle, qui fait réellement tourner les conteneurs. Un cluster en a d'un seul à plusieurs milliers.

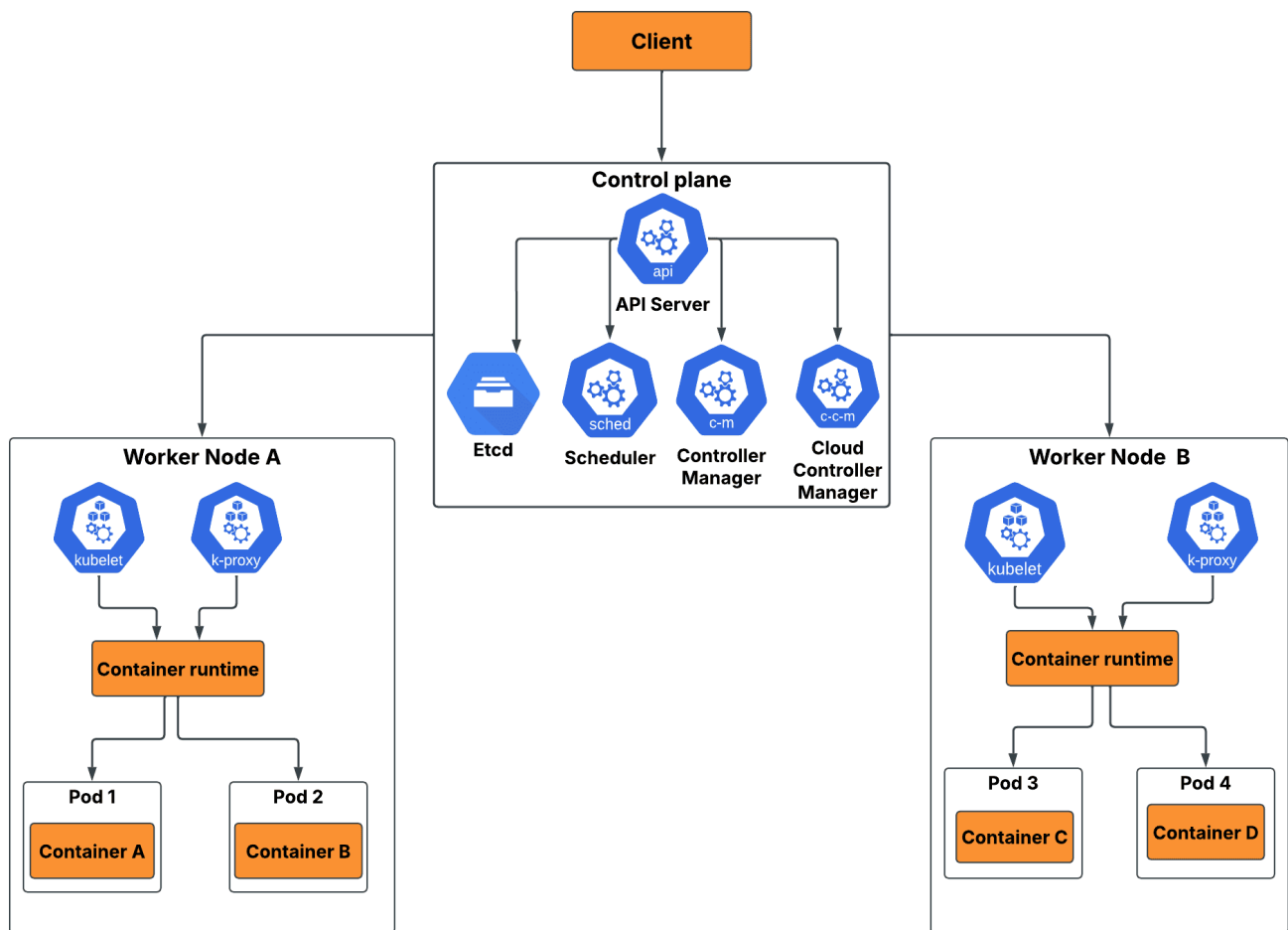
Sur chaque node, trois composants tournent, et eux non plus n'ont pas le même rôle :

- **le kubelet** : l'agent local du control plane sur cette machine précise. Il reçoit un ordre du control plane ("fais tourner ce pod ici"), l'exécute, puis remonte l'état réel ("il tourne" ou "il vient de mourir").

- **kube-proxy** : le routeur réseau du node. C'est lui qui dirige le trafic entrant vers le bon conteneur, même quand les copies changent de machine ou d'adresse IP.
- **le container runtime** : le moteur qui fait réellement tourner un conteneur, la couche technique sous Kubernetes. On l'a côtoyé toute la semaine sans le nommer ainsi : Docker en est un exemple, mais Kubernetes sait en piloter d'autres, comme [containerd](#), via une interface commune décrite par la [Container Runtime Interface](#), le contrat que tout moteur de conteneurs doit respecter pour qu'un kubelet sache lui parler.

Kubernetes architecture





Sur ce schéma, en haut le control plane, à gauche et à droite les nodes qui portent les pods. Une requête de `kubectl` entre toujours par l'api-server, jamais directement sur un node : c'est la porte unique dont on parlait plus haut.

Le voyage d'un kubectl apply, de la commande au conteneur

Reprenons la question d'ouverture, et suivons la commande pas à pas, sur notre exemple "hello" à trois copies :

1. `kubectl apply -f deployment.yaml` part du poste de la personne qui tape la commande, et atteint l'**api-server**, seule porte d'entrée du cluster.
2. L'api-server **valide** le texte reçu (un YAML bien formé, un objet reconnu), puis l'**écrit dans etcd**. À cet instant, l'état voulu existe, mais rien ne tourne encore.
3. Le **controller manager**, qui surveille etcd en continu, remarque l'écart : 3 copies voulues, 0 copie réelle. Il crée 3 objets Pod, encore vides de toute machine.
4. Le **scheduler** repère ces 3 pods sans machine assignée, et choisit, pour chacun, le node le mieux placé selon les ressources disponibles.
5. Sur chaque node choisi, le **kubelet** reçoit l'ordre "fais tourner ce pod ici", et demande au **container runtime** de tirer l'image et de démarrer le conteneur.
6. Le kubelet **remonte l'état réel** ("le pod tourne") à l'api-server, qui le réécrit dans etcd. La boucle de réconciliation compare à nouveau : 3 voulues, 3 réelles, écart nul, elle ne fait plus rien jusqu'au prochain changement.

Six étapes, un seul aller-retour côté utilisateur. Tout le reste se passe sans qu'on ait besoin d'y penser, et c'est exactement le service que rend un orchestrateur : transformer une intention en une réalité qui tient, toute seule.

Ce voyage suppose, à chaque étape, que tout se passe bien. Deux endroits précis où il peut caler méritent d'être vus dès maintenant, parce qu'ils reviennent, sous une autre forme, en section 6. À l'étape 4, si aucun node du cluster n'a assez de mémoire ou de CPU disponible pour accueillir un pod, le scheduler ne peut tout simplement pas choisir de machine : le pod reste sans affectation, visible dans un état `Pending`. À l'étape 5, si le nom de l'image est mal orthographié ou que l'image n'existe pas sur le registry, le kubelet échoue à la tirer, et le pod reste bloqué avant même d'avoir démarré un conteneur. Dans les deux cas, rien ne plante bruyamment : le cluster attend, patiemment, en répétant la même tentative, exactement comme la boucle de réconciliation de la section 2 le fait toujours.

Pour aller plus loin : ce que k3s et k3d remplacent

Un vrai [Kubernetes](#) complet, avec toutes ses pièces réparties sur plusieurs machines, pèse lourd à installer et à maintenir : plusieurs composants, souvent plusieurs serveurs rien que pour le control plane. Un excellent outil pour une vraie production à grande échelle, un mauvais choix pour apprendre ou pour un petit projet.

On a déjà croisé la réponse à ce problème au Jour 2, en une ligne : [K3S](#), une distribution allégée de Kubernetes créée par Rancher, qui tient dans un seul binaire de moins de 100 Mo. Ce n'est pas un Kubernetes au rabais, c'est un vrai Kubernetes conforme aux standards, avec trois simplifications concrètes :

- **etcd disparaît au profit de SQLite** en installation mono-serveur, une base de données bien plus simple à faire tourner qu'un etcd distribué, suffisante tant qu'un seul serveur gère le control plane.
- **Traefik est déjà installé**, un reverse proxy qui sait lire les objets Ingress qu'on manipulera en section 4, sans qu'on ait à l'installer soi-même.
- **un répartiteur de charge léger est intégré**, pour exposer un service sans dépendre d'un vrai load balancer cloud, absent sur une machine qu'on administre soi-même.

K3S s'installe très bien directement sur une machine Linux, en une commande. Mais tourner un cluster complet juste pour apprendre pose une autre question, très concrète : et si la machine de la personne qui apprend est un Mac, ou un Windows ? C'est là qu'intervient un deuxième outil, complémentaire, pas concurrent.

k3d, c'est quoi ? Un outil qui fait tourner des nodes K3S à l'intérieur de conteneurs Docker, plutôt que directement sur la machine. Chaque "node" du cluster est, en réalité, un conteneur Docker. Doc officielle : [k3d.io](#).

Ce détail change tout pour l'apprentissage : puisque Docker tourne de façon quasi identique sur Linux, macOS et Windows, un cluster k3d se comporte de la même façon partout, sans jamais toucher à l'installation système de qui que ce soit. On crée un cluster, on le détruit, on en recrée un autre, en quelques secondes, sans rien laisser derrière soi sur la machine hôte.

```
# Crée un cluster k3d nommé "demo", un seul node (le comportement par défaut,  
# sans option -a pour ajouter des agents), et mappe le port 8080 de la  
# machine hôte vers le port 80 du répartiteur de charge intégré au cluster  
k3d cluster create demo -p "8080:80@loadbalancer"
```

Devrait afficher : une série de lignes de progression (création du réseau Docker, du node, du répartiteur de charge), puis un message de succès, et un cluster prêt à recevoir des commandes `kubectl`.


```
kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
k3d-demo-server-0	Ready	control-plane,master	30s	v1.30.x+k3s1

Un seul node, `Ready`, qui joue à la fois le rôle de control plane et de node d'exécution vus dans la sous-section précédente : c'est exactement le comportement mono-machine annoncé plus haut.

Un dernier coup d'œil, avec un outil déjà connu depuis le Jour 1, rend la promesse de k3d très concrète :

```
docker ps
```

CONTAINER ID	IMAGE	NAMES
a1b2c3d4e5f6	ghcr.io/k3d-io/k3d-proxy:5.x	k3d-demo-serverlb
f6e5d4c3b2a1	rancher/k3s:v1.30.x-k3s1	k3d-demo-server-0

Le node du cluster et son répartiteur de charge ne sont, vus depuis Docker, que deux conteneurs ordinaires, listés par la commande la plus banale qui soit. Le "cluster Kubernetes" d'aujourd'hui n'est jamais qu'un Docker qu'on regarde différemment.

Ce cluster de démonstration a fait son travail, et il se rend aussi vite qu'il s'est créé. Le geste compte, parce qu'il libère le port 8080 de la machine, celui dont le vrai cluster aura besoin cet après-midi : deux clusters ne peuvent pas écouter sur le même port de l'hôte en même temps.

```
k3d cluster delete demo
```

Le `@loadbalancer` de la commande précédente cible un conteneur bien particulier, que k3d crée automatiquement devant le ou les nodes du cluster : un petit reverse proxy qui reçoit tout le trafic venu de la machine hôte, et le redirige vers le port équivalent à l'intérieur du cluster. C'est cette porte-là qui laissera entrer le trafic vers Traefik, puis vers les Ingress, en section 4.

Les deux outils ne se concurrencent pas, ils s'empilent, chacun réglant un problème que l'autre ne règle pas :

Outil	Ce qu'il fait	Le problème qu'il règle
K3S	Une distribution complète de Kubernetes, dans un seul binaire	Un vrai Kubernetes trop lourd à installer pour apprendre ou pour un petit projet
k3d	Fait tourner des nodes K3S dans des conteneurs Docker	Un cluster identique, quel que soit le système d'exploitation de la machine qui apprend

Le cluster créé plus haut ne compte qu'un seul node, celui qui joue à la fois le rôle de control plane et de node d'exécution, exactement le comportement par défaut d'un `k3d cluster create` sans options supplémentaires. Ajouter des agents (l'option `-a`, pour "agents") créerait d'autres conteneurs qui rejoindraient ce même cluster comme nodes d'exécution supplémentaires, un moyen de simuler, sur un seul laptop, un cluster à plusieurs machines. Retenez le principe, on n'en aura pas besoin cette semaine.

Le message à retenir avant de continuer : rien de ce qu'on installe ici n'est une version simplifiée qui trahirait le vrai outil. Tout ce qu'on manipule aujourd'hui, les objets, les commandes `kubectl`, le vocabulaire, se transporte tel quel sur un cluster d'entreprise, chez un fournisseur cloud ou sur des serveurs maison. Seule la taille change.

Une dernière mise en regard, avant d'attaquer le vocabulaire des objets en section 4, pour ancrer ce qui vient de changer sur ce qu'on connaissait déjà :

Ce qu'on connaissait (Docker seul)	Ce qui le remplace dans un cluster
Une seule machine, qui décide et exécute	Control plane et nodes, séparés
<code>docker run</code> , exécuté une fois	Un état déclaré, maintenu en continu
<code>docker-compose.yml</code>	Un ou plusieurs manifestes YAML
Le moteur Docker, qui fait tourner les conteneurs	Le container runtime, piloté par le kubelet

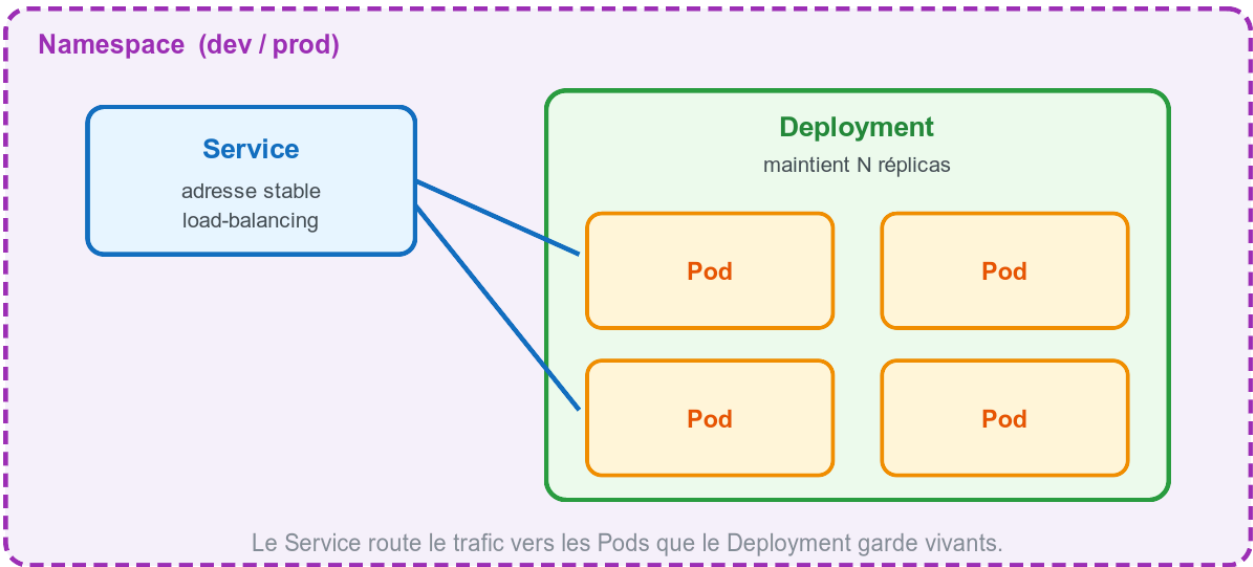
4 - Les objets qu'on manipule vraiment

Un cluster Kubernetes ne connaît qu'une seule façon de recevoir des ordres : un fichier YAML, qu'on appelle un **manifeste**, décrivant un objet précis, envoyé via `kubectl apply -f`.

Un manifeste, c'est quoi ? Un fichier, presque toujours YAML, qui décrit l'état voulu d'un objet Kubernetes : son type, son nom, sa configuration. C'est ce fichier-là que `kubectl apply -f` envoie à l'api-server. Doc officielle : [Objects In Kubernetes](#).

Sept objets suffisent à couvrir l'essentiel de ce qu'on écrira cette semaine. On les prend un par un, toujours sur des exemples neutres, un `nginx`, une application "hello", jamais sur la Todo API : elle attend cet après-midi.

Les objets Kubernetes



L'essentiel des objets

Le Pod : ce qui tourne vraiment

Un conteneur seul, lancé au milieu d'un cluster sans rien autour, pose un problème immédiat : personne ne le surveille, personne ne le relance s'il meurt, personne ne lui donne d'adresse stable. Kubernetes ne manipule donc jamais un conteneur nu, il l'enveloppe toujours dans un objet plus petit que tout le reste.

Pod ? La plus petite unité déployable dans Kubernetes. Une enveloppe autour d'un ou plusieurs conteneurs qui partagent le même réseau et le même stockage. La grande majorité du temps, un pod contient un seul conteneur : pensez "un pod, c'est une copie de mon application qui tourne". Doc officielle : [Pods](#).

Voilà le manifeste le plus court possible, un nginx tout nu :

```
# pod-nginx.yaml
apiVersion: v1          # la version de l'API Kubernetes pour ce type d'objet
kind: Pod               # le type d'objet qu'on décrit
metadata:
  name: nginx           # le nom de ce pod, unique dans son namespace
  labels:
    app: nginx          # une étiquette, on y revient à la toute fin de la section
spec:
  containers:
    - name: nginx        # le nom du conteneur à l'intérieur du pod
      image: nginx:1.27-alpine
      ports:
        - containerPort: 80  # le port que le conteneur écoute, à titre documentaire
```

```
kubectl apply -f pod-nginx.yaml
```

Devrait afficher : `pod/nginx created` . Ensuite, `kubectl get pods` va lister ce pod avec le statut `Running` une fois l'image tirée et le conteneur démarré.

Un piège attend quiconque s'arrête ici : ce pod, créé seul, sans rien pour le surveiller, ne redémarre pas s'il plante. Le supprimer avec `kubectl delete pod nginx` le fait disparaître pour de bon, personne ne le recrée. C'est très exactement le problème que l'objet suivant résout.

Un deuxième piège, plus surprenant : la quasi-totalité des champs d'un pod sont **immuables** une fois créé, l'api-server refuse purement et simplement la plupart des modifications à chaud. En pratique, personne ne retouche un pod à la main de toute façon : la bonne réponse à "je veux une nouvelle version" n'est jamais d'éditer un pod existant, c'est de laisser un objet de plus haut niveau le remplacer proprement. C'est précisément ce que l'objet suivant automatise.

Le Deployment : celui qui maintient le compte

Un pod isolé n'offre aucune des garanties vues en section 2 et 3 : pas de self-healing, pas de plusieurs copies, pas de mise à jour progressive. Il fallait un objet qui pilote des pods à notre place, en appliquant la boucle de réconciliation, plutôt que d'en créer un à la main et de croiser les doigts.

Un Deployment, c'est quoi : l'objet qui maintient un nombre choisi de pods identiques en vie, à tout instant, et qui sait aussi les mettre à jour sans coupure (section 5). On ne crée quasiment jamais de pod directement, on crée des Deployments, et ils créent les pods pour nous. Doc officielle : [Deployment](#).

```
# deployment-hello.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hello
  labels:
    app: hello
spec:
  replicas: 3                # l'état voulu de la section 2, en une seule ligne
  selector:
    matchLabels:
      app: hello            # les pods que CE Deployment considère comme les siens
  template:                 # le moule utilisé pour fabriquer chaque pod
    metadata:
      labels:
        app: hello         # les étiquettes posées sur chaque pod créé
    spec:
      containers:
        - name: hello
          image: nginxdemos/hello:latest
          ports:
            - containerPort: 80
```

```
kubectl apply -f deployment-hello.yaml
kubectl get pods
```

Devrait afficher : trois pods, nommés `hello-<hash>-<suffixe>`, tous `Running`. Supprimer l'un d'eux à la main, et un nouveau réapparaît en quelques secondes : la boucle de réconciliation vue en section 2, appliquée pour de vrai.

Le Service : une adresse qui ne bouge pas

Chaque pod du Deployment précédent porte sa propre adresse IP interne, et cette adresse change à chaque fois qu'un pod meurt et qu'un autre le remplace. Un client qui viserait directement l'IP d'un pod perdrait sa cible dès la première panne.

Le Service, c'est quoi ? Une adresse stable, à l'intérieur du cluster, qui répartit le trafic vers un groupe de pods repérés par une étiquette. Les pods vont et viennent, l'adresse du Service, elle, ne bouge jamais. Doc officielle : [Service](#).

```
# service-hello.yaml
apiVersion: v1
kind: Service
metadata:
  name: hello
spec:
  type: ClusterIP          # accessible uniquement depuis l'intérieur du cluster
  selector:
    app: hello             # cible tous les pods portant cette étiquette
  ports:
    - port: 80             # le port exposé par le Service
      targetPort: 80       # le port réellement écouté par les pods
```

```
kubectl apply -f service-hello.yaml
kubectl get svc hello
```

Devrait afficher : une ligne avec une `CLUSTER-IP` propre au Service, distincte de celle de chaque pod, et qui ne change jamais tant que le Service existe.

Ce comportement rappelle très précisément quelque chose de connu : dans un `docker-compose.yml`, un conteneur en joint un autre par son nom de service, jamais par son adresse IP, parce que le réseau interne de Compose résout ce nom tout seul. Le Service Kubernetes est le cousin direct de cette idée : un client à l'intérieur du cluster tape simplement `http://hello`, et Kubernetes résout ce nom vers l'adresse stable du Service, qui répartit ensuite vers les pods vivants.

Le `type: ClusterIP` de l'exemple ci-dessus n'est qu'une des trois formes que peut prendre un Service, celle qui reste strictement interne au cluster. Retenez simplement leurs noms pour l'instant, ils reviennent quelques sous-sections plus bas, au moment d'ouvrir le cluster sur l'extérieur : `NodePort` ouvre un port fixe sur chaque node du cluster, et `LoadBalancer` demande un vrai répartiteur de charge à un fournisseur cloud, absent sur les machines qu'on administre soi-même.

ConfigMap et Secret : la configuration hors de l'image

Coder en dur une variable d'environnement dans une image Docker oblige à reconstruire l'image entière pour changer une seule valeur, et coder en dur un mot de passe dans un manifeste versionné dans Git le rend visible de quiconque a accès au dépôt.

Mais c'est quoi, un ConfigMap ? Un objet qui stocke de la configuration non sensible, sous forme de paires clé-valeur, qu'on injecte ensuite dans un pod sans jamais toucher à l'image. Doc officielle : [ConfigMap](#).

```
# configmap-hello.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: hello-config
data:
  GREETING: "Bonjour depuis le ConfigMap" # une simple paire clé/valeur
  LOG_LEVEL: "info"
```

Un mot sur Secret : exactement la même forme qu'un ConfigMap, réservée aux données sensibles (mot de passe, jeton d'API), avec ses valeurs encodées en base64. Doc officielle : [Secret](#).

```
# secret-hello.yaml
apiVersion: v1
kind: Secret
metadata:
  name: hello-secret
type: Opaque                # un Secret générique, sans format imposé
data:
  API_KEY: c3VwZXJzZWNyZXQ= # "supersecret" encodé en base64, pas chiffré
```

Un point de sécurité qui mérite une phrase, pas plus, parce qu'on le recroisera : le base64 n'est pas du chiffrement, seulement un encodage réversible en une commande (`echo c3VwZXJzZWNyZXQ= | base64 -d`). Un Secret protège contre une lecture accidentelle dans un `kubectl get -o yaml` bête, pas contre quelqu'un qui a un accès légitime à etcd.

Injecter les deux dans un pod se fait de la même façon, une variable d'environnement qui pointe vers une clé précise :

```
# extrait d'un pod qui consomme le ConfigMap et le Secret ci-dessus
env:
- name: GREETING
  valueFrom:
    configMapKeyRef:
      name: hello-config
      key: GREETING
- name: API_KEY
  valueFrom:
    secretKeyRef:
      name: hello-secret
      key: API_KEY
```

```
kubectl apply -f configmap-hello.yaml
kubectl apply -f secret-hello.yaml
kubectl get configmap hello-config
kubectl get secret hello-secret
```

Devrait afficher : les deux objets listés par leur nom. Notez que `kubectl describe secret hello-secret` n'affiche jamais les valeurs en clair, seulement leurs clés, précisément pour éviter qu'un `describe` distrait ne les fasse défiler sur un partage d'écran.

Le ConfigMap est le cousin direct du fichier `.env` déjà manipulé depuis le Jour 1 : une configuration séparée du code, qu'on peut changer sans toucher à l'image. Le Secret suit la même logique, avec un avertissement de sécurité en plus.

Un Secret peut aussi naître directement en ligne de commande, sans passer par un fichier YAML, pratique pour un test rapide sans laisser de valeur en clair traîner sur disque :

```
kubectl create secret generic hello-secret --from-literal=API_KEY=supersecret
```

Et un dernier réflexe de sécurité, à garder pour la suite du cours : monter un Secret en volume plutôt qu'en variable d'environnement réduit sa surface d'exposition. Une variable d'environnement se propage sans effort : à tout process enfant, dans un message d'erreur qui l'affiche par mégarde, ou dans un outil de crash reporting mal configuré. Un fichier monté en volume reste, lui, exactement là où on est allé le lire.

Le PersistentVolumeClaim : les données qui survivent au pod

Un pod est **éphémère** par construction : il peut être détruit et recréé à tout instant, par une mise à jour, un redémarrage de node, ou la boucle de réconciliation elle-même. Tout ce qui vit uniquement dans le système de fichiers d'un conteneur disparaît avec lui, exactement comme un conteneur Docker sans volume attaché.

Un PersistentVolumeClaim, pour le dire simplement : une demande de stockage, exprimée par un pod, que le cluster satisfait en attachant un espace disque qui survit à la destruction du pod. On l'abrège en PVC. Doc officielle : [Persistent Volumes](#).

```
# pvc-hello.yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: hello-data
spec:
  accessModes:
    - ReadWriteOnce      # un seul node peut écrire dessus à la fois
  resources:
    requests:
      storage: 1Gi        # la taille demandée
```

Gi, Mi : ces suffixes-là ? Les unités binaires de Kubernetes pour la mémoire et le stockage. Un **Gi** (gibi-octet) vaut 1024 **Mi**, soit environ 1,07 gigaoctet, et un **Mi** (mébi-octet) vaut 1024 kibi-octets. On les retrouvera partout aujourd'hui, du stockage demandé ici jusqu'aux limites mémoire de la section 5.

```
# extrait d'un pod qui monte ce PVC dans son système de fichiers
volumeMounts:
  - name: data
    mountPath: /var/lib/hello  # le dossier, dans le conteneur, qui pointe vers le
    stockage persistant
volumes:
  - name: data
    persistentVolumeClaim:
      claimName: hello-data
```

```
kubectl apply -f pvc-hello.yaml
kubectl get pvc hello-data
```

Devrait afficher : un statut **Bound**, signe que le cluster a trouvé (ou créé) un espace de stockage réel derrière la demande. Sur k3d comme sur K3S, ce travail se fait tout seul grâce à une StorageClass installée par défaut, qui crée un dossier sur le node à chaque nouvelle demande : personne n'a besoin de préparer un disque à la main.

Ce mécanisme est le cousin direct d'un volume Docker nommé, déjà utilisé pour survivre à un `docker compose down` depuis le jour 1. La différence tient en un mot : sur un cluster à plusieurs nodes, le PVC doit en plus garantir que le pod sera reprogrammé sur le node qui porte réellement les données, ce que gère la StorageClass à notre place.

Le champ `accessModes: ReadWriteOnce` mérite un œil attentif, parce qu'il porte une contrainte silencieuse : ce volume ne peut être monté en écriture que par un seul node à la fois. Sur le cluster mono-node d'aujourd'hui, la question ne se pose jamais. Sur un cluster à plusieurs nodes, en revanche, le scheduler doit reposer le pod exactement sur le node qui porte déjà les données du PVC, jamais un autre : un stockage local, contrairement à un vrai stockage réseau, ne voyage pas d'une machine à l'autre.

L'Ingress : la porte d'entrée du cluster

Un Service `ClusterIP`, vu plus haut, ne répond qu'à l'intérieur du cluster. Pour qu'un navigateur, depuis la machine hôte, atteigne une application qui tourne dedans, il faut une porte d'entrée qui sache router le trafic externe vers le bon Service, selon le nom d'hôte demandé.

L'Ingress, c'est quoi ? Un objet qui décrit des règles de routage HTTP, "ce nom d'hôte va vers ce Service", pour le trafic qui arrive de l'extérieur du cluster. Il ne fait rien tout seul : il a besoin d'un Ingress controller pour l'exécuter, et k3s en installe un par défaut, Traefik, vu en section 3. Doc officielle : [Ingress](#).

```
# ingress-hello.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: hello-ingress
spec:
  ingressClassName: traefik      # l'Ingress controller installé par défaut avec k3s
  rules:
    - host: hello.localhost      # le nom d'hôte qui déclenche cette règle
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: hello        # le Service défini plus haut dans cette section
                port:
                  number: 80
```

```
kubectl apply -f ingress-hello.yaml
kubectl get ingress
```

Devrait afficher : une ligne pour `hello-ingress`, avec la colonne `HOSTS` affichant `hello.localhost`. Le trajet complet, depuis un navigateur, ressemble à ceci : la machine hôte tape `http://hello.localhost:8080`, ce port atteint le répartiteur de charge de k3d vu en section 3, qui le transmet à Traefik à l'intérieur du cluster, qui lit cet Ingress, reconnaît le nom d'hôte, et route vers le Service `hello`, qui répartit enfin vers un des trois pods.

Les deux autres types de Service croisés plus haut règlent le même problème, en plus rustique : `NodePort` ouvre un port fixe sur chaque node, qu'il faut connaître et retenir, et `LoadBalancer` réclame un répartiteur au fournisseur cloud, donc de l'argent et un cloud. L'Ingress, lui, tient plusieurs services derrière un seul point d'entrée, et les distingue par leur nom d'hôte.

Un seul Ingress peut d'ailleurs porter plusieurs règles à la fois, chacune avec son propre `host` ou son propre `path`, pour envoyer par exemple `hello.localhost` vers un Service et `api.localhost` vers un autre, sur ce même point d'entrée. C'est cette capacité-là qui fait de l'Ingress le vrai chef d'orchestre du trafic entrant, plutôt qu'une simple porte ouverte sur un seul Service.

Le Namespace : la cloison entre deux mondes

Tous les objets vus jusqu'ici ont atterri dans le même espace, par défaut. Sur un petit projet de cours, ça ne pose aucun problème. Sur un cluster partagé par plusieurs équipes, ou par plusieurs environnements d'un même projet, deux objets du même nom finiraient par se marcher dessus.

Namespace ? Une cloison logique à l'intérieur d'un même cluster, qui isole un groupe d'objets d'un autre. Deux objets peuvent porter le même nom, tant qu'ils vivent dans des namespaces différents. Doc officielle : [Namespaces](#).

```
# namespace-demo.yaml
apiVersion: v1
kind: Namespace
metadata:
  name: demo
```

```
kubectl apply -f namespace-demo.yaml
kubectl get pods -n demo
```

Devrait afficher : `No resources found in demo namespace.`, un namespace tout neuf, vide, prêt à recevoir des objets. Ajouter `-n demo` (ou `--namespace demo`) à n'importe quelle commande vue plus haut y déploierait un Deployment, un Service ou un PVC séparé, sans toucher à ce qui existe ailleurs.

Rien d'équivalent n'existe vraiment dans Docker Compose, où chaque projet vit déjà dans son propre réseau isolé par construction. Le rapprochement le plus juste, c'est plutôt deux dossiers de projets Compose distincts, gardés bien séparés sur la même machine : même principe d'isolation, porté au niveau du cluster entier.

Un détail de résolution de noms mérite d'être posé ici, parce qu'il explique un bug fréquent : le nom court d'un Service, `http://hello` vu plus haut, ne fonctionne que depuis un pod du même namespace. Un pod du namespace `demo` qui voudrait joindre un Service du namespace `default` doit utiliser son nom complet, `hello.default.svc.cluster.local`, ou au minimum `hello.default`. Un `http://hello` qui répondait hier depuis `default` et qui échoue soudainement depuis un nouveau namespace n'est presque jamais un bug réseau, seulement l'effet de cette cloison qu'on vient de poser.

L'essentiel de kubectl : les commandes qui servent vraiment

Toutes les commandes croisées jusqu'ici ne sont qu'un sous-ensemble de kubectl, vu en section 3. Voilà celles qui reviennent tous les jours, au-delà de `apply` et `get` déjà vues :

- `kubectl describe <type> <nom>` : le détail complet d'un objet, et surtout sa section `Events` en bas, qui raconte ce que le cluster a tenté et ce qui a échoué. C'est le tout premier réflexe face à un objet qui ne se comporte pas comme prévu, creusé en section 6.
- `kubectl logs <pod>` : la sortie standard du conteneur, exactement ce qu'un `docker logs` donnerait. Ajouter `--previous` récupère les logs de la tentative précédente, celle d'avant un redémarrage, indispensable pour comprendre pourquoi un pod vient de crasher (section 6 encore).
- `kubectl exec -it <pod> -- /bin/sh` : un shell interactif à l'intérieur d'un conteneur en cours d'exécution, le cousin direct d'un `docker exec -it`.
- `kubectl port-forward <pod> 8080:80` : redirige un port de la machine locale vers un port d'un pod précis, sans passer par un Service ni un Ingress. Pratique pour un test rapide, jamais une solution d'exposition durable.
- `kubectl rollout status deployment/<nom>` et `kubectl rollout undo deployment/<nom>` : suivre une mise à jour en cours, et revenir en arrière si elle tourne mal. Le cœur de la section 5.
- `kubectl scale deployment/<nom> --replicas=N` : change à la volée le nombre de copies voulues, sans toucher au manifeste sur disque.
- `kubectl delete -f <fichier>` : supprime tout ce que le fichier décrit, le geste inverse d'un `apply`.
- `kubectl get events --sort-by=.lastTimestamp` : la liste de tout ce qui s'est passé récemment dans un namespace, toutes ressources confondues, triée dans l'ordre chronologique. Utile quand on ne sait même pas quel objet regarder en premier.

Deux d'entre elles valent la peine d'être vues tourner, sur le Deployment "hello" créé plus haut :

```
# Un shell interactif dans le premier pod venu du Deployment "hello"
kubectl exec -it hello-6d4cf56db6-7xkzq -- sh
```

Devrait afficher : une invite de terminal qui change, `/ #` ou équivalent, signe qu'on est désormais "dedans" le conteneur, exactement comme un `docker exec -it` sur une machine unique.

```
# Redirige le port 8080 de la machine locale vers le port 80 du pod
kubectl port-forward hello-6d4cf56db6-7xkzq 8080:80
```

Devrait afficher : `Forwarding from 127.0.0.1:8080 -> 80`, et un `curl localhost:8080` depuis un autre terminal répond avec la page du pod ciblé, celui-là précisément, sans passer par le Service qui répartirait normalement vers les deux autres.

Instant Cheat Sheet !

Commande	Ce qu'elle fait
<code>kubectl apply -f fichier.yaml</code>	Crée ou met à jour un objet depuis un manifeste
<code>kubectl get <type></code>	Liste les objets d'un type donné
<code>kubectl describe <type> <nom></code>	Détail complet, events compris

Commande	Ce qu'elle fait
<code>kubectl logs <pod> [--previous]</code>	Logs du conteneur, ou de sa tentative précédente
<code>kubectl exec -it <pod> -- sh</code>	Shell interactif dans un conteneur
<code>kubectl port-forward <pod> 8080:80</code>	Tunnel local vers un pod, pour un test rapide
<code>kubectl rollout status/undo deployment/<nom></code>	Suivre ou annuler une mise à jour
<code>kubectl scale deployment/<nom> -- replicas=N</code>	Change le nombre de copies voulues
<code>kubectl delete -f fichier.yaml</code>	Supprime ce que le fichier décrit
<code>kubectl get events</code>	Tout ce qui s'est passé récemment dans le namespace

On pousse plus loin : un manifeste décortiqué champ par champ

Reprenons le manifeste du Deployment "hello" vu plus haut, et regardons-le une deuxième fois, pas pour ce qu'il fait, mais champ par champ, parce que c'est ce schéma exact qui revient dans tous les manifestes qu'on écrira cette semaine.

```

apiVersion: apps/v1      # (1)
kind: Deployment         # (2)
metadata:                # (3)
  name: hello
  labels:
    app: hello
spec:                    # (4)
  replicas: 3
  selector:              # (5)
    matchLabels:
      app: hello
  template:               # (6)
    metadata:
      labels:
        app: hello
    spec:
      containers:
        - name: hello
          image: nginxdemos/hello:latest

```

- **(1) `apiVersion`** : la version de l'API Kubernetes concernée par ce type d'objet. Un Pod utilise `v1`, un Deployment utilise `apps/v1` : chaque type d'objet a la sienne, et se tromper dessus fait échouer la validation avant même que quoi que ce soit ne soit créé.

kind	apiVersion
Pod	v1
Deployment	apps/v1
Service	v1
ConfigMap / Secret	v1
PersistentVolumeClaim	v1
Ingress	networking.k8s.io/v1
Namespace	v1

Pas de logique magique à mémoriser, seulement une habitude à prendre : `v1` couvre les objets présents depuis les tout débuts de Kubernetes, les objets plus récents ou plus spécialisés (Deployment, Ingress) vivent dans un groupe d'API nommé, séparé du cœur historique.

- **(2) kind** : le type d'objet décrit, un des sept vus dans cette section.
- **(3) metadata** : l'identité de l'objet lui-même, son nom, ses étiquettes propres. C'est par ce nom que `kubectl get deployment hello` le retrouve.
- **(4) spec** : la spécification, le cœur du texte, ce qu'on veut vraiment. Son contenu change radicalement d'un type d'objet à l'autre, mais le mot-clé reste toujours le même.
- **(5) selector.matchLabels** : la règle que ce Deployment utilise pour reconnaître "ses" pods, parmi tous les pods du cluster.
- **(6) template** : le moule utilisé pour fabriquer chaque nouveau pod, avec ses propres `metadata.labels`, en apparence redondants avec le `selector` juste au-dessus.

Cette redondance apparente entre `selector.matchLabels` et `template.metadata.labels` n'en est pas une, et c'est le détail qui casse le plus souvent une première tentative de manifeste. Le `selector` dit "je gère les pods qui portent cette étiquette", le `template` dit "les pods que je crée porteront cette étiquette-ci". Si les deux ne coïncident pas, l'api-server refuse carrément la création, avec un message explicite :

```
The Deployment "hello" is invalid: spec.template.metadata.labels: Invalid value:
map[string]string{"app":"hello-v2"}: `selector` does not match template `labels`
```

Un Deployment qui générerait des pods sans jamais pouvoir les retrouver n'aurait aucun sens, c'est exactement ce que cette règle empêche. Retenez le réflexe : un `selector` et un `template.metadata.labels` identiques, toujours, sur chaque Deployment qu'on écrira aujourd'hui comme cet après-midi.

5 - Mettre à jour sans couper, et encaisser la charge

Le déploiement d'hier remplaçait un conteneur par un autre, en deux commandes, avec une coupure de quelques secondes assumée entre les deux. Un cluster fait mieux, et c'est là que le nombre de copies vu en section 4 commence à vraiment payer.

L'essentiel du rolling update

Comment remplacer trois pods par leur nouvelle version, sans jamais faire tomber le nombre de pods disponibles à zéro ?

Rolling update, en une ligne : une mise à jour qui remplace les pods un par un, jamais tous d'un coup, en gardant toujours suffisamment de pods de l'ancienne version vivants tant que la nouvelle n'est pas prête. C'est la stratégie par défaut d'un Deployment, documentée sur la même page officielle que le Deployment lui-même.

Deux réglages pilotent le rythme de ce remplacement :

- **maxSurge** : combien de pods en plus de l'objectif le Deployment a le droit de créer, temporairement, pendant la mise à jour.
- **maxUnavailable** : combien de pods peuvent manquer, temporairement, par rapport à l'objectif, pendant la même mise à jour.

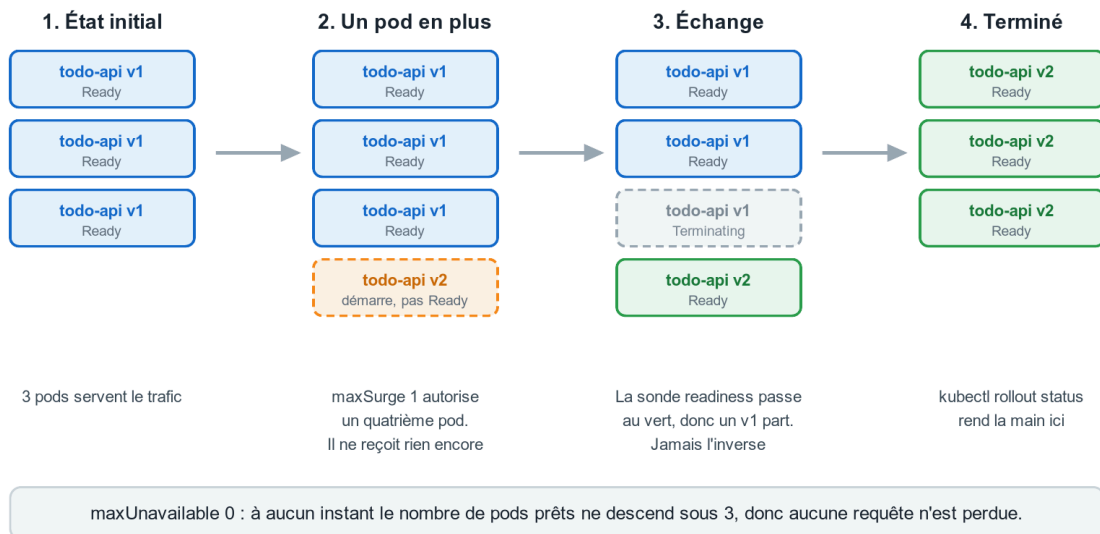
Suivons un Deployment à 3 replicas, avec `maxSurge: 1` et `maxUnavailable: 0`, le réglage le plus prudent, celui qui interdit toute baisse de capacité :

Étape	Pods v1	Pods v2	Total disponible
Avant la mise à jour	3	0	3
Un pod v2 démarre (le surplus autorisé)	3	1	4
Il devient prêt, un pod v1 est retiré	2	1	3
Un nouveau pod v2 démarre	2	2	4
Il devient prêt, un pod v1 est retiré	1	2	3
Dernier pod v2 démarre, puis prêt	0	3	3

À chaque instant, au moins 3 pods répondent. Jamais 0, jamais même en dessous de 3. C'est le prix du `maxSurge: 1`, un pod de plus consommé temporairement, contre zéro coupure.

Rolling update : 3 replicas, maxSurge 1, maxUnavailable 0

Le Deployment remplace les pods un par un, et le Service n'envoie du trafic qu'aux pods prêts



Le réglage inverse existe aussi, `maxSurge: 0` et `maxUnavailable: 1` : aucun pod supplémentaire n'est jamais créé, mais la capacité peut redescendre à 2 pods sur 3 pendant la bascule. Plus rapide à écrire, moins gourmand en ressources temporaires, au prix d'une capacité réduite le temps du remplacement. Le choix entre les deux dépend d'une seule question : une baisse de capacité momentanée est-elle acceptable, ou faut-il payer le pod en plus pour l'éviter complètement ?

Ces deux réglages se déclarent dans le manifeste du Deployment lui-même, un bloc `strategy` de plus à côté du `spec` déjà vu :

```
# extrait du Deployment "hello", avec sa stratégie de mise à jour explicite
spec:
  replicas: 3
  strategy:
    type: RollingUpdate      # le comportement par défaut, rendu explicite ici
    rollingUpdate:
      maxSurge: 1            # au plus un pod de plus que l'objectif
      maxUnavailable: 0     # jamais un pod de moins que l'objectif
```

Sans ce bloc, Kubernetes applique déjà des valeurs par défaut raisonnables (`maxSurge: 25%`, `maxUnavailable: 25%`), suffisantes pour un premier essai, mais qui méritent d'être rendues explicites dès qu'une coupure, même partielle, devient inacceptable.

```
# Change l'image du Deployment, ce qui déclenche automatiquement le rolling update
kubectl set image deployment/hello hello=nginxdemos/hello:v2

# Suit la progression en direct, ligne par ligne
kubectl rollout status deployment/hello
```

Devrait afficher : une série de lignes du type `waiting for deployment "hello" rollout to finish: 1 out of 3 new replicas have been updated...`, jusqu'à `deployment "hello" successfully rolled out`.

Et si la version v2 est cassée une fois en place, revenir à la précédente ne demande ni script ni recherche du bon tag, une seule commande suffit :

```
kubectl rollout undo deployment/hello
```

Devrait afficher : le même mécanisme de rolling update, mais dans l'autre sens, jusqu'à ce que les 3 pods soient revenus à l'ancienne version.

Et si la version v2 ne plante pas, mais ne devient jamais prête ? Un dernier cas qui piège souvent une première mise à jour, avant de passer à la suite. Le rolling update reste bloqué à mi-chemin, un pod v2 créé, jamais promu, jamais retiré, `kubectl rollout status` qui n'avance plus. Pas de panique dans ce cas-là, juste une attente qui ne se résoudra jamais toute seule : la commande de retour arrière vue plus haut reste valable à tout moment, y compris en plein milieu d'une mise à jour bloquée. Ce qui décide qu'un pod est "prêt" ou non, c'est justement le sujet qui vient.

L'essentiel des sondes : readiness et liveness

Le Jour 1, un `HEALTHCHECK` dans le Dockerfile disait à Docker comment vérifier, à intervalle régulier, qu'un conteneur répondait vraiment, pas juste qu'il avait démarré. Les sondes de Kubernetes reprennent exactement cette idée, un cran au-dessus : ce n'est plus Docker qui surveille un conteneur isolé, c'est le cluster tout entier qui surveille chaque pod, et qui réagit différemment selon la sonde qui échoue.

Deux sondes existent, et elles ne répondent pas à la même question :

La readiness probe, en une ligne : "ce pod est-il prêt à recevoir du trafic, maintenant ?" Si elle échoue, le pod est retiré des Services qui le ciblent, sans être redémarré. Il reste vivant, simplement mis de côté le temps qu'il se rétablisse. Doc officielle : [Liveness, Readiness and Startup Probes](#).

Et la liveness probe ? "Ce pod est-il toujours vivant, ou faut-il le redémarrer ?" Si elle échoue, Kubernetes tue le conteneur et en redémarre un nouveau à sa place.

```
# extrait du Deployment "hello", avec ses deux sondes sur une route de santé
readinessProbe:
  httpGet:
    path: /health
    port: 80
  periodSeconds: 5          # vérifie toutes les 5 secondes

livenessProbe:
  httpGet:
    path: /health
    port: 80
  initialDelaySeconds: 10   # laisse 10 secondes à l'app pour démarrer
  periodSeconds: 10
```

Le cas qui distingue vraiment les deux sondes, et qui piège presque tout le monde la première fois : une application qui répond correctement au endpoint de santé, mais dont la logique métier est cassée derrière. La readiness probe, elle, continue de répondre 200, puisqu'elle n'interroge que la route `/health`, pas les fonctionnalités réelles de l'app. Le pod reste marqué prêt, reçoit du trafic, et sert des erreurs à chaque vraie requête, sans que Kubernetes ne remarque rien. Une sonde n'est jamais meilleure que ce qu'elle vérifie réellement : un `/health` qui répond "200, toujours" sans jamais tester une vraie dépendance (la base de données, par exemple) donne une fausse tranquillité.

Et sans aucune sonde déclarée, un pod n'est pas nu pour autant : Kubernetes le considère prêt dès que son conteneur a démarré, et vivant tant que son process principal tourne. Une application qui met dix secondes à se connecter à sa base avant de pouvoir répondre recevrait donc du trafic dès la première seconde, en l'absence de readiness probe pour le retenir en coulisses le temps qu'il faut.

Approfondissons : replicas, requests et limits

Le nombre de copies d'un Deployment, `replicas`, fixe combien de pods tournent, mais ne dit rien de ce que chaque pod a le droit de consommer. Sans indication, un pod peut, en théorie, engloutir toute la mémoire ou tout le CPU disponible sur son node, au détriment des autres pods qui y vivent.

Autrement dit, pour `requests` et `limits` : `requests` dit au scheduler "ce pod a besoin d'au moins ça pour tourner", et sert à choisir le node. `limits` dit au node "ce pod ne dépassera jamais ça", et une fois franchi côté mémoire, le conteneur est tué. Doc officielle : [Resource Management for Pods and Containers](#).

```
# extrait d'un conteneur avec ses requests et limits déclarées
resources:
  requests:
    cpu: "250m"          # un quart de cœur CPU, le minimum garanti
    memory: "128Mi"
  limits:
    cpu: "500m"          # jamais plus d'un demi cœur
    memory: "256Mi"      # dépasser cette valeur en mémoire tue le conteneur
```

Trois cas à distinguer, selon ce qu'on déclare ou pas :

- **`requests` et `limits` tous deux posés** : le scheduler place le pod en connaissance de cause, et le node protège les voisins d'un pod qui dérape.
- **`requests` sans `limits`** : le pod peut consommer bien plus que prévu, au risque d'affamer les autres pods du même node.
- **rien du tout** : le scheduler place le pod sans aucune garantie de ressources. C'est le pire des trois cas sur un node partagé, où un seul pod gourmand peut faire chuter tous les autres.

Le troisième cas raconte une panne classique, celle du voisin bruyant : sur un node qui porte plusieurs pods sans `limits` déclarées, un seul pod en boucle de calcul intensive peut saturer tout le CPU disponible. Les autres pods du même node continuent d'exister, mais chacune de leurs réponses ralentit, jusqu'à dépasser le délai de la readiness probe vue plus haut. Ces pods, pourtant en parfaite santé applicative, se retrouvent retirés des Services les uns après les autres, uniquement parce qu'un voisin mal élevé leur a volé leur temps CPU. Poser des `limits` sur chaque conteneur, même généreuses, évite précisément ce scénario : un pod ne peut jamais faire chuter les autres, il ne peut faire chuter que lui-même.

Une dernière notion mérite d'être nommée, sans qu'on s'y attarde aujourd'hui : le **scaling horizontal automatique**, ou [Horizontal Pod Autoscaler](#), ajuste tout seul le nombre de `replicas` selon la charge mesurée (le CPU consommé, par exemple), en s'appuyant justement sur les `requests` posées plus haut. Retenez le nom, la mécanique se creusera à un autre moment du cours.

Rolling update, sondes, `requests` et `limits` : le cluster sait désormais mettre à jour une application sans la couper, et se protéger d'un pod qui déraile. Tout ce mécanisme suppose, implicitement, que le cluster ait de quoi communiquer clairement ce qui se passe. Reste une question qu'on a repoussée à chaque détour de cette section : quand ça se passe mal quand même, un pod qui refuse de démarrer, une sonde qui échoue en boucle, comment le lire, vite et juste ? C'est exactement le sujet qui vient.

6 - Lire un cluster qui va mal

Imaginons qu'on ait trois copies demandées. `kubectl get pods` en affiche deux `Running` et une bloquée sur un statut inconnu. Rien n'explose bruyamment, aucune alerte ne sonne toute seule : juste un pod qui refuse de se comporter comme les autres, et rien d'autre à se mettre sous la main que quelques commandes.

L'essentiel du dépannage

describe et events : là où le cluster dit ce qu'il n'arrive pas à faire

Face à un pod qui ne tourne pas, par où commencer, avant même de lire une seule ligne de log ?

`kubectl describe pod <nom>` reste, dans l'immense majorité des cas, le tout premier réflexe. Pas pour son résumé de configuration, en haut, mais pour sa section `Events`, tout en bas, qui raconte chronologiquement ce que le cluster a tenté et ce qui a échoué.

```
Events:
  Type      Reason      Age   From          Message
  ----      -
  Normal    Scheduled   30s   default-scheduler   Successfully assigned demo/hello-abc to
node1
  Normal    Pulling     29s   kubelet        Pulling image "monpseudo/hello:v9"
  Warning   Failed      28s   kubelet        Failed to pull image: not found
  Warning   BackOff     20s   kubelet        Back-off pulling image "monpseudo/hello:v9"
```

Quatre lignes, et le diagnostic est déjà posé : l'image `monpseudo/hello:v9` n'existe pas là où le cluster va la chercher. Aucun besoin de fouiller les logs applicatifs, puisque l'application n'a jamais eu la chance de démarrer, le problème se situe avant elle.

`kubectl logs <pod>` prend le relais dès qu'un pod a réussi à démarrer au moins une fois : c'est la sortie standard du conteneur, identique à ce qu'un `docker logs` donnerait. Un détail change tout après un crash : `kubectl logs <pod> --previous` récupère les logs de la tentative *précédente*, celle d'avant le redémarrage. Sans cet indicateur, `kubectl logs` montre les logs du conteneur tout juste relancé, encore vierges, et la vraie cause du crash reste invisible dans les logs de la tentative morte.

Prenons un deuxième cas, plus retors que le précédent, parce que l'image se tire correctement et que le pod démarre bel et bien, avant de tomber tout seul. `kubectl get pods` affiche un `CrashLoopBackOff`, avec un compteur de redémarrages qui grimpe :

NAME	READY	STATUS	RESTARTS	AGE
hello-9f7c2	0/1	CrashLoopBackOff	4	3m

Le réflexe `kubectl logs hello-9f7c2`, tout court ne montre rien d'utile ici : le conteneur vient tout juste de redémarrer pour la cinquième fois, et sa sortie est encore vide. C'est `--previous` qui referme le dossier :

```
kubectl logs hello-9f7c2 --previous
```

```
Error: connect ECONNREFUSED 127.0.0.1:5432
at TCPConnectWrap.afterConnect [as oncomplete]
```

L'application tente de joindre une base de données sur `127.0.0.1`, l'adresse locale du pod lui-même, alors que la base tourne ailleurs, dans un autre pod. En cause, ici : une variable d'environnement mal renseignée, ou carrément absente. Le `describe pod` d'un peu plus haut n'aurait rien montré de tel, puisque ses events racontent ce que le cluster a tenté de faire, jamais ce que l'application fait une fois lancée. C'est précisément la frontière entre les deux commandes, `describe` pour tout ce qui empêche un pod de démarrer, `logs --previous` pour tout ce qui le fait tomber une fois démarré.

Les états d'un pod, leur cause et leur remède

Cinq statuts reviennent sans arrêt, et chacun raconte une histoire différente :

État	Ce qu'il signifie	Où regarder
<code>Pending</code>	Le pod existe dans etcd, mais aucun node ne peut encore l'accueillir	<code>describe pod</code> , section Events : ressources insuffisantes, ou aucun node prêt
<code>ImagePullBackOff</code>	Le kubelet n'arrive pas à récupérer l'image indiquée	Le nom de l'image, son tag, et si le registry est bien public ou accessible
<code>CrashLoopBackOff</code>	Le conteneur démarre, plante, et Kubernetes réessaie en boucle, avec un délai croissant	<code>kubectl logs --previous</code> , presque toujours une variable manquante ou une dépendance injoignable
<code>OOMKilled</code>	Le conteneur a dépassé sa <code>limits.memory</code> , et le node l'a tué	La <code>limits.memory</code> vue en section 5, souvent trop basse pour la charge réelle
<code>Evicted</code>	Le node lui-même manquait de ressources, et a expulsé le pod pour se protéger	La pression globale sur le node, <code>kubectl describe node</code> , pas seulement le pod

Deux d'entre eux se ressemblent au premier coup d'œil et se confondent souvent : `OOMKilled` sanctionne un conteneur précis qui a dépassé sa propre limite, `Evicted` sanctionne un pod parce que le node entier manquait de ressources, indépendamment de ce que ce pod-là consommait. Le premier se corrige en ajustant une `limits`, le second en regardant l'ensemble des pods du node, pas seulement le coupable apparent.

Le premier de la liste, `Pending`, mérite un dernier détour, parce qu'il ne dit jamais lui-même sa cause exacte, seulement que le scheduler n'a pas encore réussi à placer le pod :

```
Events:
  Type      Reason             Age   From                  Message
  ----      -
Warning    FailedScheduling   10s   default-scheduler     0/1 nodes are available:
                                1 Insufficient memory.
```

La cause, ici, tient dans le message même : une `requests.memory` demandée par le pod, plus haute que ce que le seul node du cluster a de disponible. Baisser la `requests`, ou libérer de la mémoire ailleurs sur le node, sont les deux seules issues, le pod restera `Pending` indéfiniment sinon, la boucle de réconciliation continuant de réessayer sans jamais y parvenir.

Une dernière commande complète ce tableau : [metrics-server](#), un composant que k3s installe déjà tout seul, alimente `kubectl top nodes` et `kubectl top pods`, qui affichent la consommation CPU et mémoire réelle, en direct, sans passer par Prometheus.

```
kubectl top pods
```

NAME	CPU (cores)	MEMORY (bytes)
hello-9f7c2	142m	198Mi
hello-a1b3c	98m	201Mi
hello-c7d9e	2000m	512Mi

La troisième ligne saute aux yeux : `2000m` de CPU consommé et `512Mi` de mémoire, largement au-dessus des deux autres pods pourtant identiques. C'est exactement le voisin bruyant de la section 5, repéré en une commande, avant même qu'un `OOMKilled` ne vienne sanctionner quelqu'un d'autre. Un premier réflexe rapide, avant de sortir l'artillerie plus lourde de la section suivante.

Pour les curieux : brancher le Prometheus d'hier sur le cluster

Le Prometheus installé hier scrapait une application qui tournait sur une seule machine, à une adresse fixe, écrite en dur dans `prometheus.yml`. Un cluster casse cette hypothèse d'un coup : les pods naissent, meurent, changent d'adresse IP à chaque redémarrage, et un fichier de configuration figé deviendrait faux en quelques minutes.

La solution ne consiste pas à réécrire `prometheus.yml` plus souvent, mais à laisser Prometheus interroger directement le cluster pour savoir, en temps réel, quels pods scraper. Prometheus tourne alors lui-même comme un pod à l'intérieur du cluster, avec l'autorisation de questionner l'api-server, et il découvre ses cibles dynamiquement plutôt que de les lire dans une liste figée. Chaque fois qu'un pod apparaît ou disparaît, la liste des cibles se met à jour toute seule, sans qu'un humain touche à un fichier de configuration.

Concrètement, ce mécanisme s'appuie sur les mêmes étiquettes que celles vues en section 4 : Prometheus demande à l'api-server la liste des pods qui portent une étiquette convenue, et scrape leur `/metrics` sans jamais connaître leur adresse IP à l'avance. Un pod qui redémarre change d'adresse, garde son étiquette, et reste visible de Prometheus sans qu'un fichier n'ait besoin d'être touché.

Un dernier nom mérite d'être glissé ici, sans qu'on s'y arrête : [kube-state-metrics](#), un service qui interroge l'api-server et transforme l'état des objets Kubernetes eux-mêmes (le nombre de replicas voulu contre le nombre réel, un pod bloqué en `Pending` depuis trop longtemps) en métriques Prometheus. Là où `/metrics` racontait la santé d'une application, kube-state-metrics raconte la santé du cluster qui la porte.

On retrouve, une fois branché, exactement le même vocabulaire que celui construit hier : les mêmes métriques, les mêmes labels, les mêmes quatre golden signals sur un tableau de bord Grafana. Seule la façon dont Prometheus trouve ses cibles a changé, le reste du raisonnement tient tel quel.

`describe`, `logs --previous`, les cinq états du tableau, `kubectl top` : on sait maintenant lire ce qu'un cluster raconte de ses propres pannes. Reste un dernier document à mettre à jour avec tout ce vocabulaire neuf, celui qu'on suit à l'aveugle en pleine nuit, la procédure de déploiement écrite hier. Elle parlait d'une machine. Il est temps de la faire parler d'un cluster.

7 - La procédure de déploiement quand la cible est un cluster

La procédure écrite hier décrivait une machine précise, `vm-prod`, avec ses commandes SSH, son conteneur nommé, son port fixe. Cette après-midi, la cible change de nature : ce n'est plus une machine, c'est un cluster qui applique lui-même une partie de ce que la procédure décrivait à la main.

L'essentiel de ce qui change

Le squelette d'une bonne procédure, vu hier, ne bouge pas : prérequis et accès, versions concernées, étapes numérotées avec vérification après chacune, retour arrière, contacts et escalade, durée estimée, vérifications avant et après. Ce qui change, c'est le contenu de plusieurs de ces rubriques précises, pas leur existence.

- **Le retour arrière** devient une seule commande, `kubectl rollout undo deployment/<nom>`, là où hier il fallait redéployer l'image du sha précédent à la main.
- **La vérification après déploiement** s'appuie sur `kubectl rollout status`, qui attend que le rolling update soit terminé, complétée par les sondes readiness vues en section 5, qui écartent d'elles-mêmes un pod qui répondrait mal.
- **La liste des pannes connues** s'enrichit des états de pod vus en section 6, `ImagePullBackOff`, `CrashLoopBackOff`, `OOMKilled`, `Evicted`, chacun avec son propre remède, à ajouter à côté des `Permission denied` et `Connection timed out` d'hier.
- **La ligne "relancer le service" disparaît purement et simplement.** Le cluster l'a déjà fait, avant même qu'un humain n'ait eu le temps de le remarquer, exactement le scénario de 3h du matin qui a ouvert la journée.
- **La durée estimée et la fenêtre de maintenance** se réduisent, sans disparaître complètement. Le rolling update de la section 5 vise zéro coupure visible, la fenêtre "où couper le service" que la procédure d'hier devait négocier n'a donc plus lieu d'être pour un déploiement qui se passe bien. Elle reste utile pour une chose précise : le temps que prendrait un retour arrière si tout tournait mal, ce chiffre-là ne disparaît jamais d'une bonne procédure.

En un coup d'œil, la même bascule, rubrique par rubrique :

Rubrique de la procédure	Hier (la cible est <code>vm-prod</code>)	Aujourd'hui (la cible est un cluster)
Retour arrière	Redéployer l'image du sha précédent, à la main	<code>kubectl rollout undo deployment/<nom></code>

Rubrique de la procédure	Hier (la cible est <code>vm-prod</code>)	Aujourd'hui (la cible est un cluster)
Vérification	<code>curl</code> sur le port de l'app, une fois	<code>kubectl rollout status</code> , puis les sondes <code>readiness</code>
Pannes connues	<code>Permission denied</code> , <code>Connection timed out</code>	<code>ImagePullBackOff</code> , <code>CrashLoopBackOff</code> , <code>OOMKilled</code> , <code>Evicted</code>
Redémarrage manuel	Une étape à part, "relancer le service"	Disparaît, le cluster l'a déjà fait
Fenêtre de coupure	Négociée à l'avance, quelques minutes	Réduite à la durée d'un rollback éventuel

Creusons : la procédure écrite pour une machine qui n'existe plus

Voilà le diff exact entre deux fragments de la même procédure, avant et après, sur les rubriques qui bougent vraiment :

Avant (la cible est `vm-prod`) :

```
### Retour arrière
Commande : ssh deploy@vm-prod "docker stop todo-api && docker rm todo-api && \
  docker run -d --name todo-api -p 3000:3000 monpseudo/todo-api:<sha-precedent>"
Vérification : curl vm-prod:3000/health doit répondre 200

### Pannes connues
- Permission denied (publickey) : mauvaise clé SSH côté pipeline
- Connection timed out : le pare-feu bloque le port 22
```

Après (la cible est le cluster) :

```
### Retour arrière
Commande : kubectl rollout undo deployment/todo-api -n todo
Vérification : kubectl rollout status deployment/todo-api -n todo, puis
  curl todo.localhost:8080/health doit répondre 200 sur chaque pod

### Pannes connues
- ImagePullBackOff : le tag d'image n'existe pas sur le registry
- CrashLoopBackOff : kubectl logs --previous pour voir la vraie cause
- OOMKilled : la limits.memory du Deployment est trop basse
```

Rien n'a changé dans la structure du document, rien n'a changé dans l'esprit de la règle posée hier, une commande exacte, un critère de vérification précis, jamais une phrase vague. Ce qui a changé, c'est uniquement ce que ces commandes ciblent.

L'anti-pattern à éviter tient en une phrase, et il guette précisément aujourd'hui : une procédure qui continuerait de parler de `vm-prod`, de `docker restart`, d'un port SSH précis, alors que la cible réelle est devenue un cluster qui gère tout ça lui-même. Une procédure qui décrit une machine qui n'existe plus ne dégrade pas seulement la qualité du document, elle envoie la personne d'astreinte taper des commandes sur

un système qui ne répond plus de cette façon-là, au pire moment pour le découvrir.

8 - À votre tour : du serveur unique au cluster (*après-midi*)

Ce matin, le cluster était un exemple neutre : une application "hello" qu'on scale, qu'on tue pour la regarder revenir, qu'on met à jour sans coupure. Cet après-midi, c'est le même geste sur votre propre Todo API, celle qui tournait hier sur une seule machine, avec un seul point de défaillance et personne pour la relancer à 3h du matin.

La machine d'hier, `vm-prod`, a rendu un service précis : prouver qu'un push suffit à déployer, rien de plus. Les trois limites posées ce matin, la coupure à chaque déploiement, l'absence de relève à 3h du matin, la seule copie pour encaisser le trafic, n'ont toujours pas bougé. Douze phases plus tard, elles seront fausses toutes les trois.

Le projet

À la fin de l'après-midi, le dépôt et le cluster racontent ceci :

- un cluster **todo-cluster** tourne sur votre poste, avec la Todo API dedans, dans son propre namespace,
- la configuration et les secrets vivent hors de l'image, dans des objets que seul le cluster connaît,
- la base PostgreSQL survit à la suppression de son pod, parce que ses données ne vivent plus dans le conteneur,
- un `git push` sur `main` met le cluster à jour tout seul, et la pipeline refuse de continuer si le rollout ne converge pas,
- trois copies de l'API se partagent le trafic, et deux sondes distinctes décident qui le reçoit et qui redémarre,
- une panne tirée au hasard se répare parfois toute seule, parfois pas, et le dépôt garde la trace de ce qui distingue les deux,
- la procédure de déploiement d'hier parle maintenant de rollout et de replicas, elle ne mentionne plus SSH.

L'objectif tient en une phrase : *une machine qui tombe ne coupe plus rien, et un push suffit pour que le cluster le sache.*

Palier 1 : l'application tourne dans le cluster

Phase 1 : le cluster, et la Todo API dessus

Avant de commencer : le dépôt se note sur ses commits autant que sur son résultat. Un commit par phase terminée, des fichiers ajoutés un par un, jamais de `git add .` en aveugle.

`vm-prod` a bien rendu service hier, mais elle occupe le port 3000 et une part de la RAM dont le cluster a besoin pour démarrer. Premier geste de la journée, donc : `docker stop vm-prod`. Elle n'est pas supprimée, seulement mise en pause, elle redeviendra utile le jour où il faudra comparer les deux mondes.

Place au cluster. [k3d](#) fait tourner [K3s](#) (la distribution légère de Kubernetes vue ce matin) à l'intérieur de conteneurs Docker, ce qui donne un cluster complet sans machine virtuelle ni installation lourde. L'installation tient en une ligne :

```
curl -s https://raw.githubusercontent.com/k3d-io/k3d/main/install.sh | bash
```

Sous Windows ou macOS ? k3d a besoin d'un Docker qui tourne, comme tout ce qu'on fait depuis trois jours. Si l'installation de k3d échoue sur votre machine, [Docker Desktop embarque son propre cluster Kubernetes](#) mono-nœud : il se crée depuis la vue *Kubernetes* du tableau de bord, bouton *Create cluster*, en choisissant le type *Kubeadm*. Les commandes `kubectl` qui suivent fonctionnent à l'identique dessus, seule la création du cluster change.

Le cluster naît avec son port d'entrée déjà mappé, celui qui servira dès la phase 4 :

```
# Si le cluster de démonstration de ce matin tourne encore, il tient le port 8080 :
# k3d cluster delete demo le libère, et la création ci-dessous passe du premier coup
k3d cluster create todo-cluster -p "8080:80@loadbalancer"
```

Le `@loadbalancer` de cette ligne, croisé ce matin, désigne le petit répartiteur que k3d place devant le cluster : le port 8080 de votre machine ressort sur le port 80 du cluster, très exactement le chemin qu'un Ingress empruntera en phase 4.

Devrait afficher, une fois le cluster prêt :

```
$ kubectl get nodes
NAME                                STATUS    ROLES                                AGE    VERSION
k3d-todo-cluster-server-0          Ready    control-plane,master                30s    v1.30.x+k3s1
```

Un namespace range ce qui va suivre à l'écart du reste :

```
kubectl create namespace todo
```

Reste la partie qui compte : un Deployment et un Service pour votre propre image, celle que la pipeline pousse sur Docker Hub depuis hier. La structure ci-dessous tient le contrat de la journée (namespace `todo`, noms `todo-api`, port 3000 dans le conteneur), le reste est à écrire :

```
# todo-api-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: todo-api
  namespace: todo
spec:
  replicas: 1
  selector:
    matchLabels:
      app: todo-api
  template:
    metadata:
      labels:
        app: todo-api
    spec:
      containers:
        - name: todo-api
```

```
# TODO : l'image, VOTRE_PSEUDO/todo-api, avec le tag d'un sha précis (jamais latest)
image:
ports:
  - containerPort: 3000
```

```
# todo-api-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: todo-api
  namespace: todo
spec:
  type: ClusterIP
  # TODO : selector qui cible les pods portant le label app: todo-api
  # TODO : port 80 exposé par le Service, targetPort 3000 (celui du conteneur)
```

Personne n'est bloqué par une image manquante ou cassée : un `nginx:alpine`, ou n'importe quelle image publique connue, peut prendre la place de la Todo API le temps de vérifier que le Deployment et le Service fonctionnent. La vraie image reprend sa place dès qu'elle répond sur Docker Hub.

```
kubectl apply -f todo-api-deployment.yaml
kubectl apply -f todo-api-service.yaml
kubectl get pods -n todo
```

Sans Ingress pour l'instant, un `kubectl port-forward` suffit à vérifier que l'API répond depuis l'intérieur du cluster :

```
kubectl port-forward -n todo svc/todo-api 3000:80
curl -s localhost:3000/health
```

Trois vérifications avant d'enchaîner :

- `kubectl get pods -n todo` montre un pod `Running`, jamais `Pending` ni `ImagePullBackOff`,
- supprimer ce pod à la main (`kubectl delete pod`) en fait réapparaître un autre en quelques secondes, sans qu'on ait rien retapé : c'est la boucle de réconciliation vue ce matin, pour la première fois sur votre propre application,
- une image dont le nom contient une faute de frappe laisse le pod bloqué en `ImagePullBackOff`, jamais en `Running` : le cluster ne fait pas semblant que ça marche.

Phase 2 : la configuration sort de l'image

Côté sécurité, tout de suite : cette phase fabrique un objet Secret. Rien de ce qu'il contient ne doit apparaître ailleurs, ni dans un `echo` de débogage, ni dans un message de commit.

L'image Docker Hub de votre API ne connaît ni l'hôte de la base, ni son mot de passe : hier, ces valeurs vivaient dans un `.env` copié à la main sur `vm-prod`. Le cluster, lui, les distribue par deux objets déjà croisés ce matin sur un exemple neutre, un ConfigMap pour ce qui n'est pas sensible et un Secret pour ce qui l'est.


```
# todo-config.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: todo-config
  namespace: todo
data:
  # TODO : les variables non sensibles que votre API lit au démarrage
  # (l'hôte et le port de la base, l'environnement...)
```

Une différence avec le Secret vu ce matin saute aux yeux dans le fichier ci-dessous : il s'écrit avec `stringData` plutôt que `data`. Kubernetes accepte alors du texte en clair et se charge lui-même de l'encoder en base64 au moment de l'enregistrer, ce qui évite de le faire à la main pour chaque valeur.

```
# todo-secret.yaml
apiVersion: v1
kind: Secret
metadata:
  name: todo-secret
  namespace: todo
type: Opaque
stringData:
  # TODO : DB_USER et DB_PASSWORD, les deux variables que votre code exige au démarrage
```

Reste à brancher les deux sur le conteneur. `envFrom` déverse toutes les clés d'un objet dans l'environnement du conteneur d'un coup, sans les nommer une par une.

`envFrom` ? Un champ de la spec d'un conteneur qui injecte, comme variables d'environnement, l'intégralité des clés d'un ConfigMap (`configMapRef`) ou d'un Secret (`secretRef`). Utile ici, exactement parce que votre API attend déjà `DB_HOST`, `DB_USER`, `DB_PASSWORD` sous ces noms depuis le Jour 1.

```
containers:
- name: todo-api
  image: VOTRE_PSEUDO/todo-api:<sha>
  ports:
    - containerPort: 3000
    # TODO : envFrom, qui référence todo-config (configMapRef) et todo-secret (secretRef)
```

Devrait afficher, une fois le pod relancé avec ce changement : à l'intérieur du conteneur, les variables `DB_HOST`, `DB_USER`, `DB_PASSWORD` existent, et pourtant aucune de ces trois valeurs ne se trouve dans l'image ni dans le dépôt Git.

Un `kubectl exec -n todo <pod> -- env` le confirme directement. La vraie question, plus intéressante, se pose en trois temps :

- le cas normal marche quand les deux objets existent et que le pod redémarre après leur création,
- le cas limite apparaît si le pod n'a jamais redémarré depuis : un ConfigMap modifié ne pousse rien tout seul vers un pod déjà vivant, il faut le relancer pour qu'il relise son environnement,

- le cas cassant, c'est un Secret volontairement vide de toute clé : le pod démarre quand même, simplement privé des variables absentes, et c'est le code de l'API qui doit refuser de tourner sans elles plutôt que de se connecter à une base au hasard.

Phase 3 : la base et ses données qui survivent

Côté dépôt : les manifestes de cette phase, comme ceux d'avant, se commitent tels quels. Rien dans ce paragraphe ne demande de secret à écrire en clair dans un fichier versionné.

Un pod PostgreSQL sans volume perd tout à chaque redémarrage, ce qui suffisait pour un exercice mais jamais pour une vraie base. La gestion d'une base en cluster n'est pas l'objectif du jour, alors la voici prête, complète, à commiter sans y toucher :

```
# todo-db.yaml : la base PostgreSQL, prête à l'emploi
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: todo-db-data
  namespace: todo
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: todo-db
  namespace: todo
spec:
  replicas: 1                # une base ne se duplique pas comme un service sans état
  selector:
    matchLabels:
      app: todo-db
  template:
    metadata:
      labels:
        app: todo-db
    spec:
      containers:
        - name: postgres
          image: postgres:16-alpine
          env:
            - name: POSTGRES_DB
              valueFrom:
                secretKeyRef:
                  name: todo-secret
                  key: DB_NAME
            - name: POSTGRES_USER
              valueFrom:
                secretKeyRef:
```

```

        name: todo-secret
        key: DB_USER
    - name: POSTGRES_PASSWORD
      valueFrom:
        secretKeyRef:
          name: todo-secret
          key: DB_PASSWORD
  ports:
    - containerPort: 5432
  volumeMounts:
    - name: data
      mountPath: /var/lib/postgresql/data
      # subPath évite un dossier "lost+found" que certains volumes déposent
      # à la racine, et que Postgres refuse au démarrage
      subPath: pgdata
  volumes:
    - name: data
      persistentVolumeClaim:
        claimName: todo-db-data
---
apiVersion: v1
kind: Service
metadata:
  name: todo-db
  namespace: todo
spec:
  selector:
    app: todo-db
  ports:
    - port: 5432
      targetPort: 5432

```

Ce fichier suppose que `todo-secret` contient déjà une clé `DB_NAME` en plus des deux de la phase précédente : c'est le moment de la rajouter si elle manque encore. k3d fournit sa propre classe de stockage par défaut, `local-path` (un volume provisionné automatiquement sur le disque du nœud), donc aucune ligne `storageClassName` n'est nécessaire pour que la PVC trouve un volume.

Reste à compléter le `DB_HOST` de `todo-config`, resté vide en phase 2. Sa valeur n'est ni une adresse IP ni `localhost` : c'est `todo-db`, le nom exact du Service ci-dessus. Le DNS interne du cluster, vu ce matin, résout ce nom vers les pods qui portent l'étiquette `app: todo-db`, quel que soit leur nombre ou leur IP du moment.

```

kubectl apply -f todo-db.yaml
kubectl get pvc -n todo

```

Devrait afficher :

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES	AGE
todo-db-data	Bound	pvc-xxxxxx	1Gi	RWO	10s

Le vrai test ne regarde pas si la base démarre, il regarde si elle se souvient :

- créer une tâche via l'API, puis supprimer le pod `todo-db` (pas le Deployment, juste le pod) et attendre qu'il en revienne un : la tâche est toujours là au prochain `GET`,
- couper le Service `todo-db` en modifiant son selector par erreur : l'API ne trouve plus la base par son nom, alors même que les pods PostgreSQL tournent toujours,
- supprimer carrément la PVC pendant que le Deployment `todo-db` tourne encore : la suppression reste bloquée, parce qu'un volume attaché à un pod vivant ne se libère pas au hasard.

Phase 4 : la porte d'entrée

Un mot sur les commits : l'Ingress de cette phase est un fichier de plus dans le dépôt, pas une configuration tapée une fois puis oubliée. Il se committe avec le reste.

Jusqu'ici, seul `kubectl port-forward` permettait d'atteindre l'API depuis l'extérieur du cluster, une commande à retaper à chaque fois et jamais ce qu'un vrai utilisateur ferait. [Traefik](#), le contrôleur d'entrée que K3s embarque par défaut, écoute déjà sur le port 80 du cluster, celui que `-p "8080:80@loadbalancer"` a relié à votre machine dès la phase 1. Reste à lui dire quel nom de domaine envoyer vers quel Service, avec un objet Ingress vu ce matin sur un exemple neutre.

```
# todo-ingress.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: todo-ingress
  namespace: todo
spec:
  ingressClassName: traefik
  rules:
    # TODO : un hôte, todo.localhost, et un chemin "/" qui pointe vers
    # le Service todo-api sur son port 80
```

Devrait afficher, une fois l'Ingress appliqué :

```
kubectl get ingress -n todo
curl -s -H "Host: todo.localhost" http://localhost:8080/health
```

```
{"status":"ok","timestamp":"..."}
```

L'en-tête `Host` dans le `curl` évite de dépendre de la résolution DNS de `todo.localhost` sur votre machine, qui varie d'un système à l'autre. Un navigateur, lui, résout `todo.localhost` tout seul : `http://todo.localhost:8080` s'ouvre directement, sans rien configurer.

Qu'est-ce qui casse si...

- l'Ingress déclare un chemin `/api` au lieu de `/`, et qu'on appelle `/health` ? Traefik répond `404`, pas parce que l'API est en panne, mais parce que la règle de routage ne couvre pas cette route,
- le `port` de la règle vise 3000 au lieu de 80, le port réellement exposé par le Service `todo-api` ? Traefik ne trouve rien à cette adresse et renvoie une erreur de passerelle, jamais une réponse de l'API,

- deux Ingress différents déclarent le même hôte `todo.localhost` avec des chemins qui se recouvrent ? Traefik en choisit un, en silence, et c'est exactement le genre d'ambiguïté qu'on évite en production en donnant un hôte distinct à chaque service.

Palier 2 : un push, et le cluster se met à jour

Phase 5 : la pipeline déploie sur le cluster

Rappel notation : cette phase ne crée aucun nouveau secret côté GitHub, et c'est précisément ce qui la rend intéressante à commiter avec un message qui l'explique.

Le job `deploy` d'hier ouvrait une connexion SSH vers une machine injoignable autrement. Aujourd'hui, la cible est le cluster qui tourne sur ce même poste, celui où `./run.sh` écoute déjà depuis hier : le runner self-hosted et `todo-cluster` vivent sur la même machine.

Kubeconfig ? Le fichier qui dit à `kubectl` quel cluster joindre et avec quels droits, en général sous `~/.kube/config`. `k3d cluster create` y a déjà fusionné les identifiants de `todo-cluster` au moment de sa création.

Le job n'a donc besoin ni de clé, ni de secret GitHub supplémentaire pour parler au cluster : juste de `kubectl` sur le PATH du runner, qui lit le même kubeconfig que celui utilisé pour toutes les commandes tapées à la main depuis ce matin.

Le job garde la même charpente qu'hier, `needs: [build]`, `runs-on: self-hosted`, déclenché sur `main` seulement. Ce qui change, c'est ce qu'il exécute :

```
deploy:
  needs: [build]
  runs-on: self-hosted
  if: github.ref == 'refs/heads/main'
  steps:
    - uses: actions/checkout@v4
      # TODO : mettre à jour l'image du Deployment todo-api avec le tag du sha
      # qui vient d'être poussé (kubectl set image, ou un kubectl apply sur un
      # manifeste dont le tag a été substitué)
      # TODO : kubectl rollout status sur ce Deployment, avec un délai raisonnable,
      # qui fait échouer le job si le rollout ne converge pas
```

Devrait afficher, une fois le job vert :

```
deployment "todo-api" successfully rolled out
```

Trois scénarios à passer :

- un push sur `main` fait apparaître, quelques secondes plus tard, un pod portant le nouveau sha dans `kubectl describe deployment todo-api -n todo`, sans qu'aucune commande n'ait été tapée à la main,
- un push sur une branche de travail construit et teste l'image, mais ne touche jamais au cluster,
- un `kubectl set image` pointé vers un tag qui n'existe pas sur Docker Hub laisse le rollout bloqué, `rollout status` ne rend jamais la main dans le délai fixé, et le job devient rouge : c'est exactement le comportement voulu, la pipeline refuse de déclarer un succès qu'elle n'a pas.

Phase 6 : trois replicas, et la preuve que le trafic se répartit

Ce qui se note ici : un `replicas: 3` qui n'a jamais été vérifié par du trafic réel ne vaut pas mieux qu'un `replicas: 1` qu'on croit sur parole.

`replicas: 1` a suffi jusqu'ici pour vérifier que le mécanisme marche, mais une seule copie encaisse tout le trafic seule et tombe avec lui. Passer à trois change la donne : le `selector.matchLabels` du Deployment, déjà écrit en phase 1, retrouve automatiquement les nouveaux pods sans qu'on touche au Service, exactement le rôle des étiquettes vu ce matin.

```
# Dans todo-api-deployment.yaml : replicas: 3
kubectl apply -f todo-api-deployment.yaml
kubectl get pods -n todo -l app=todo-api
```

Prouver que le Service répartit vraiment le trafic, et pas seulement qu'il existe trois pods, demande un peu de génération de charge. Voici le script, dépendance zéro, qui servira encore en phase 8 :

```
#!/bin/sh
# charge.sh : charge continue sur l'API, via l'Ingress, pendant N secondes.
# Usage : ./charge.sh <secondes>
DURATION="${1:-30}"
URL="http://localhost:8080/api/tasks"
END=$(( $(date +%s) + DURATION ))
TOTAL=0
FAILED=0
while [ "$(date +%s)" -lt "$END" ]; do
    CODE=$(curl -s -o /dev/null -w '%{http_code}' -H "Host: todo.localhost" "$URL")
    TOTAL=$((TOTAL + 1))
    if [ "$CODE" != "200" ]; then
        FAILED=$((FAILED + 1))
        echo "requete $TOTAL : code $CODE"
    fi
    sleep 0.1
done
echo "Total : $TOTAL requetes, $FAILED echouees (code != 200)"
```

Pendant que `charge.sh 20` tourne dans un terminal, un second terminal interroge directement chaque pod, en contournant le Service pour ne pas se mentir à soi-même :

```
for pod in $(kubectl get pods -n todo -l app=todo-api -o name); do
    kubectl port-forward -n todo "$pod" 3001:3000 &
    PF_PID=$!
    sleep 1
    echo "== $pod =="
    curl -s localhost:3001/metrics | grep http_requests_total
    kill "$PF_PID"
done
```

Avant / après, à comparer :

- avant `charge.sh`, les trois `/metrics` affichent `http_requests_total` proche de zéro sur chaque pod,

- après, les trois comptent quelque chose, dans des ordres de grandeur voisins : c'est la preuve que le Service, et pas un seul pod chanceux, a reçu la charge,
- un `selector` de Service mal recopié (une étiquette qui ne colle à aucun pod) donne l'image inverse : trois pods `Running`, et pourtant `charge.sh` ne compte que des échecs, parce que le Service n'a plus aucun endpoint vers qui router.

Phase 7 : les sondes, dont celle qui ment

Petit rappel de méthode : les probes touchent uniquement le manifeste du Deployment. Un commit qui les mélange à un autre changement se relit mal dans trois semaines.

Ce matin, la différence entre readiness et liveness restait un exemple sur l'application "hello". Ici, sans sonde câblée, un pod `Running` reste `Running` même quand `/health` répond faux, et le Service continue de lui envoyer du trafic. `/health` existe depuis hier, et c'est justement sa simplicité qui va poser problème dans un instant.

```
containers:
  - name: todo-api
    image: VOTRE_PSEUDO/todo-api:<sha>
    ports:
      - containerPort: 3000
      # TODO : readinessProbe, httpGet sur /health port 3000, avec un délai
      # initial court : elle retire le pod du Service tant qu'il n'est pas prêt
      # TODO : livenessProbe, httpGet sur /health port 3000, avec un délai
      # initial plus large : elle redémarre le conteneur s'il ne répond plus
```

Devrait afficher, une fois les deux sondes en place et le pod stabilisé : `kubectl describe pod` sans le moindre événement `Unhealthy`, et `kubectl get pods` toujours à `1/1` en colonne `READY`.

Le moment qui compte arrive en coupant la base sans toucher à l'API : `kubectl scale deployment/todo-db -n todo --replicas=0`. Le pod `todo-api` reste `Running`, `1/1`, aucune sonde ne se déclenche, et pourtant un `GET /api/tasks` renvoie une erreur 500. `/health` répond `ok` sans jamais avoir interrogé la base, donc les deux sondes, bâties dessus, mentent exactement de la même façon : elles confirment que le serveur HTTP écoute, pas que l'application rend le service attendu. Cette limite se documente, elle ne se corrige pas à la légère : un `/health` qui interroge la base à chaque appel protège d'un mensonge, mais expose l'application à une cascade de sondes qui échouent toutes en même temps si la base ralentit seulement un peu.

Qu'est-ce qui se passe si...

- tout va bien, base comprise ? Les deux sondes passent, `READY` reste à `1/1`, et `kubectl describe pod` ne montre aucun événement `Unhealthy`,
- la base est coupée, comme à l'instant ? Les sondes mentent, `READY` reste à `1/1`, et seule une requête réelle sur `/api/tasks` révèle l'écart entre ce que le pod prétend et ce qu'il rend,
- le `readinessProbe` cible un port jamais exposé par le conteneur, 3001 au lieu de 3000, par une faute de frappe ? Le pod ne passe jamais `Ready`, et le Service, sans le moindre pod prêt à router vers lui, refuse toute connexion plutôt que de répondre lentement.

Phase 8 : le rolling update sous charge, mesuré

Avant de lancer : cette phase alimente directement le Journal de bord, la section du `README.md` qui garde, jour après jour, les chiffres mesurés, avec ici des valeurs qui n'existaient pas hier. Le commit qui suit garde le tableau rempli, pas une capture d'écran.

Le rolling update et ses réglages `maxSurge` / `maxUnavailable` sont connus depuis ce matin. Restait à les voir vraiment à l'œuvre, sous charge, avec un chiffre en face plutôt qu'une intuition. `maxUnavailable: 0` interdit qu'un ancien pod parte avant qu'un nouveau soit prêt, au prix d'un `maxSurge` qui autorise une copie de plus le temps de la transition.

```
spec:
  # TODO : strategy.rollingUpdate, avec maxSurge et maxUnavailable choisis pour
  # ne perdre aucune requête pendant la mise à jour
  replicas: 3
```

Le protocole : `charge.sh 30` démarre dans un terminal, et pendant qu'il tourne, un `kubectl set image` change le tag de l'image dans un second terminal, suivi d'un `kubectl rollout status` qui affiche la progression pod par pod.

Le tableau qui suit rejoint le Journal de bord, avec les chiffres d'hier déjà notés à côté :

Déploiement	Requêtes échouées	Secondes d'indisponibilité	Temps de convergence totale
Hier, SSH manuel			
Aujourd'hui, rolling update			

Trois manières de vérifier que le nombre inscrit dans la colonne "Requêtes échouées" n'est pas un hasard : relancer `charge.sh` une deuxième fois sur le même déploiement et retrouver un total proche du premier ; remettre `maxUnavailable` à sa valeur par défaut et observer que le compteur d'échecs remonte, la preuve que le réglage précédent servait à quelque chose ; et couper `charge.sh` avant la fin du rollout, pour vérifier que le `rollout status` du job de pipeline attend bien la convergence complète, pas seulement le début de la mise à jour.

Phase 9 : le retour arrière, chronométré contre celui d'hier

Dernier rappel : c'est maintenant qu'on se compare à l'astreinte d'hier, pendant que ses chiffres sont encore frais dans le Journal de bord.

`kubectl rollout undo`, vu ce matin, retrouve ici la même urgence qu'une vraie astreinte : un tag cassé part en production, et il faut revenir en arrière avant que trop de requêtes échouent. Le déclencheur est volontaire, une image qui casse une route ou qui ne démarre pas, poussée par un `kubectl set image` direct.

```
kubectl rollout undo deployment/todo-api -n todo
kubectl rollout status deployment/todo-api -n todo
```


Le chronomètre démarre au moment où la régression est constatée (un `curl` qui renvoie une erreur, pas le moment où elle a été poussée) et s'arrête quand `/health` répond de nouveau normalement.

Ce qui doit être vrai à la fin :

- le temps mesuré ici se compare directement à celui d'hier, noté dans le Journal de bord au sortir de l'astreinte, pour la même opération, un retour à la version qui marchait,
- `kubectl rollout history deployment/todo-api -n todo` liste plusieurs révisions, et `rollout undo --to-revision=N` cible n'importe laquelle d'entre elles, pas seulement la précédente,
- un `rollout undo` lancé alors qu'aucun déploiement n'a encore eu lieu échoue proprement, avec un message qui dit qu'il n'y a rien à annuler, jamais un cluster laissé à moitié dans un état.

Palier 3 : ce que le cluster ne répare pas

Phase 10 : cinq pannes, dont deux qu'il absorbe tout seul

Avant d'écrire quoi que ce soit : cette phase se joue seul, sur votre propre cluster. Personne n'attend le résultat de personne, et le tableau produit ici est à vous.

Le cluster corrige tout seul ce que la boucle de réconciliation sait corriger, et rien de plus : un pod qui meurt revient, une configuration cassée par construction ne se répare jamais toute seule. `chaos.sh` tire une panne au hasard parmi cinq, et n'affiche rien de ce qu'il vient de faire : le sujet, c'est le diagnostic, pas la surprise. Le lire à l'avance ne triche pas, la difficulté reste de reconnaître laquelle des cinq est en train de se produire.

```
#!/bin/sh
# chaos.sh : tire une panne au hasard parmi cinq, sur todo-cluster.
# A lancer depuis votre poste, kubectl doit déjà pointer sur todo-cluster.
N=$(( $(od -An -N1 -tu1 /dev/urandom) % 5 + 1 ))
echo "$N" | base64 > .incident # la reponse, pour le debriefing seulement
POD=$(kubectl get pods -n todo -l app=todo-api -o jsonpath='{.items[0].metadata.name}')
IMAGE=$(kubectl get deployment todo-api -n todo -o jsonpath='{.spec.template.spec.containers[0].image}')
REPO="${IMAGE%%:*}"

case "$N" in
  1) kubectl delete pod -n todo "$POD" ;; # un pod disparaît
  2) kubectl exec -n todo "$POD" -- kill 1 ;; # le processus
    meurt à l'intérieur du conteneur, sans que le pod disparaisse
  3) kubectl set image deployment/todo-api todo-api="$REPO:ce-tag-n-existe-pas" -n todo ;; # l'image ciblée
    n'existe nulle part
  4) kubectl patch secret todo-secret -n todo --type=json \
    -p='[{"op":"remove","path":"/data/DB_PASSWORD}]'
    kubectl rollout restart deployment/todo-api -n todo ;; # l'appli redémarre
    sans un secret qu'elle exige
  5) kubectl patch deployment todo-api -n todo --type=json \
    -p='[{"op":"add","path":"/spec/template/spec/containers/0/resources","value":
    {"limits":{"memory":"8Mi"}}}]' ;; # la limite
    mémoire ne laisse plus l'appli démarrer
```

```
esac >/dev/null 2>&1
```

Le tableau à remplir, une ligne par panne, à partir de ce qu'un `kubectl get pods`, un `kubectl describe pod` et les événements en bas de cette sortie racontent :

Panne	Signature dans <code>kubectl get pods</code>	Signature dans <code>kubectl describe / events</code>	Se répare seule ?	Remède
Pod supprimé				
Processus tué dans le conteneur				
Tag d'image inexistant				
Clé du Secret supprimée				
Limite mémoire trop basse				

`base64 -d .incident` donne le numéro tiré une fois le diagnostic posé, pas avant. Deux lignes du tableau se répareront sans qu'on touche à rien, en quelques secondes ; les trois autres attendront une commande précise, celle notée dans la colonne remède.

Phase 11 : la procédure, version cluster

Rappel notation : `docs/PROCEDURE_DEPLOIEMENT.md` reste le même fichier qu'hier, mis à jour, jamais dupliqué en `PROCEDURE_K8S.md` à côté.

La procédure d'hier décrivait un `docker compose up -d` sur une machine unique et un retour arrière qui rejouait un tag précédent. Plus rien de tout ça ne s'applique tel quel : le déploiement passe par une pipeline qui gate sur `rollout status`, et le retour arrière est une commande `kubectl`, pas un redémarrage de conteneur.

Ce que la nouvelle version doit contenir, au minimum :

- l'accès attendu pour intervenir : `kubectl` configuré sur `todo-cluster`, quel contexte, quel namespace,
- les étapes d'un déploiement manuel d'urgence, si la pipeline elle-même est en panne, avec un point de vérification observable après chacune,
- la commande de retour arrière et son critère de déclenchement, remplaçant celle d'hier,
- les cinq pannes de la phase 10, leur signature et leur remède, reprises telles quelles dans le tableau : c'est ce qui transforme un document en outil de diagnostic plutôt qu'en archive,
- la limite découverte en phase 7 : `/health` ne garantit pas que la base répond, seulement que le serveur écoute.

Trois façons de la mettre à l'épreuve :

- la relire à voix haute, étape par étape, en se demandant à chacune si un inconnu saurait taper la commande sans deviner un nom de fichier ou de namespace,

- imaginer un cas qu'elle ne couvre pas encore, par exemple un `kubectl` qui pointe sur le mauvais cluster faute de contexte précisé, et vérifier si le document laisse le lecteur sans issue,
- suivre la procédure au mot près sur un incident tiré par `chaos.sh` sans en connaître le numéro à l'avance : chaque commande qui manque, chaque signature absente du tableau, se corrige immédiatement dans le fichier.

Phase 12 : jusqu'où on peut serrer les ressources

Petit rappel de méthode : cette phase n'a pas de fin propre, seulement une meilleure mesure que la précédente. Un commit par valeur essayée garde la trace de ce qui a été tenté, pas seulement du réglage final.

`resources.requests` et `resources.limits`, vus ce matin, n'ont encore jamais été écrits pour de vrai sur `todo-api`. Sans eux, un pod peut consommer toute la mémoire disponible du nœud et affamer ses voisins ; avec des valeurs trop généreuses, le cluster réserve pour rien. Entre les deux, il y a une valeur qui tient, et elle ne se devine pas, elle se mesure.

metrics-server, croisé ce matin, est ce qui alimente `kubectl top`. Sur un cluster k3d, rien à installer : k3s le fournit d'office, la mesure est disponible immédiatement.

```
kubectl top pods -n todo
```

Sur le cluster de secours de Docker Desktop, metrics-server n'est pas fourni, et une seule ligne le pose avant que `kubectl top` réponde :

```
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml
```

```
containers:
  - name: todo-api
    # TODO : resources.requests (cpu, memory) : ce que le pod réserve au placement
    # TODO : resources.limits (cpu, memory) : ce que le pod ne doit jamais dépasser
```

Le protocole se répète en boucle, sans jamais s'arrêter tout seul : serrer `limits.memory` un peu plus, appliquer, regarder ce que `kubectl top pods -n todo` mesure juste avant la casse, relancer `charge.sh` pendant que le pod tourne, et noter s'il tient ou finit `OOMKilled`. Dès qu'il casse, revenir un cran en arrière et vérifier qu'un rolling update, déclenché avec ce réglage final, ne fait toujours tomber aucune requête sous charge, la même preuve qu'en phase 8, cette fois avec des ressources serrées plutôt que généreuses.

Personne n'a besoin d'ajuster automatiquement le nombre de copies en fonction de la charge pour aujourd'hui : le sujet de cette phase, c'est la limite d'une seule copie, pas leur multiplication.

Deux lectures, à quelques Mébiotets d'écart, à comparer :

```
$ kubectl top pods -n todo
NAME                                CPU(cores)   MEMORY(bytes)
todo-api-7d4b9c8f5-2xk9p           12m          38Mi

$ kubectl get pods -n todo
NAME                                READY   STATUS    RESTARTS   AGE
todo-api-7d4b9c8f5-2xk9p           1/1     Running   0           4m
```

```
$ kubectl get pods -n todo
NAME                                READY   STATUS             RESTARTS   AGE
todo-api-7d4b9c8f5-2xk9p          0/1     CrashLoopBackOff   3 (20s ago) 4m

$ kubectl describe pod todo-api-7d4b9c8f5-2xk9p -n todo | grep -A1 "Last State"
    Last State:  Terminated
      Reason:    OOMKilled
```

La première paire dit qu'il reste de la marge (38Mi mesurés, une limite fixée au-dessus). La seconde dit qu'on vient de la dépasser : la colonne `RESTARTS` grimpe, et `describe` nomme la cause dans son `Last State`. Entre les deux, une seule valeur a changé, `limits.memory`, et c'est elle qui devient le chiffre à écrire dans le Journal de bord, avec la charge exacte sous laquelle elle a été trouvée : la même valeur, testée sans charge, ne prouve rien.

Une troisième lecture referme la boucle : au réglage juste en dessous de celui qui casse, `charge.sh` relancé pendant un rolling update ne doit compter aucun échec, exactement comme en phase 8. Un réglage qui tient au repos mais laisse tomber des requêtes pendant une mise à jour n'est pas encore la bonne valeur.

9 - Ce qui est rendu

Le dépôt Git de la Todo API contient, à la fin de la journée :

- les manifestes du cluster : Deployment, Service, ConfigMap, Secret, PVC, Ingress, dans un dossier `k8s/` ou équivalent,
- les workflows `.github/workflows/`, dont le job `deploy` qui gate sur `kubectl rollout status`,
- `docs/PROCEDURE_DEPLOIEMENT.md`, mis à jour pour un déploiement en cluster,
- le tableau de diagnostic des cinq pannes de la phase 10, rempli,
- un `README.md` à jour, avec la section **Journal de bord** qui gagne les entrées du jour : le tableau de comparaison de la phase 8, le retour arrière chronométré de la phase 9, et les valeurs de ressources essayées en phase 12.

Ni la clé privée d'hier, ni un `.env`, ni le fichier `kubeconfig` de `todo-cluster` ne se trouvent dans le dépôt : un kubeconfig committé donne les pleins pouvoirs sur le cluster à quiconque le lit.

Grille d'évaluation :

Critère	Poids	Ce qui est regardé
Déploiement automatisé sur le cluster	25 %	un push sur <code>main</code> déclenche un rollout, le job échoue proprement s'il ne converge pas
Manifestes du cluster	20 %	Deployment, Service, ConfigMap, Secret, Ingress, complets, cohérents entre eux, rien en dur dans l'image
Résilience mesurée	20 %	tableau de comparaison de la phase 8 rempli, retour arrière chronométré et comparé à hier
Sondes et diagnostic de pannes	15 %	readiness et liveness correctement définies, tableau des cinq pannes complet et juste

Critère	Poids	Ce qui est regardé
Procédure de déploiement, version cluster	10 %	mise à jour et non dupliquée, retour arrière à jour, pannes connues listées
Rigueur du dépôt	10 %	commits atomiques, aucun secret ni kubeconfig versionné

Apprécié en plus, sans être attendu : une valeur de `limits.memory` justifiée par un chiffre plutôt qu'une intuition, un `/health` repensé qui distingue "le serveur répond" de "la base répond", un second Ingress pour exposer une version différente de l'API en parallèle, une procédure qui a survécu à un second passage de `chaos.sh` sans être réécrite.

Récapitulatif Jour 4

Ce qu'on a appris aujourd'hui

1. **Un cluster répare ce qu'il peut voir, pas ce qu'on lui cache** : un pod supprimé revient tout seul, une image introuvable ou un secret manquant restent en panne jusqu'à ce qu'une main humaine corrige la cause.
2. **La configuration quitte l'image pour de bon** : ConfigMap et Secret, injectés par `envFrom`, remplacent le `.env` copié à la main d'hier, et une même image tourne désormais contre n'importe quelle base sans être reconstruite.
3. **Une base a besoin d'un volume, une API n'en a pas besoin** : la PVC détache les données du cycle de vie du pod, et c'est tout le sens du choix `replicas: 1` pour `todo-db` contre `replicas: 3` pour `todo-api`.
4. **Le runner n'a pas bougé, la cible si** : le même self-hosted runner qui parlait en SSH hier parle en `kubectl` aujourd'hui, parce qu'il partage la machine du cluster, ce qui rend le déploiement plus simple qu'hier, pas plus compliqué.
5. **Readiness et liveness ne protègent que ce qu'elles vérifient** : deux sondes câblées sur un `/health` trop simple laissent un pod "sain" continuer de recevoir du trafic pendant que la base est injoignable.
6. **Un rolling update se mesure, il ne se suppose pas** : `maxSurge` et `maxUnavailable` transforment une intuition en un nombre de requêtes perdues, comparable ligne à ligne avec le déploiement SSH d'hier.
7. **Serrer les ressources a une limite, et elle se trouve par l'échec** : `OOMKilled` n'est pas un bug à éviter à tout prix aujourd'hui, c'est le signal qui dit où s'arrêter.

Points d'attention pour demain

- **Gardez `todo-cluster` et son runner en état de marche** : le projet de groupe démarre dessus, personne n'a envie de tout reconstruire à 9h.
- **Relisez votre tableau de la phase 10** : demain, plusieurs services tourneront ensemble sur le même cluster, et une panne sur l'un peut en cacher une autre.
- **Le projet de groupe se joue à plusieurs services, sous charge, devant la classe** : une petite application multi-services, déployée sur vos clusters, devra tenir un incident tiré au hasard en fin de journée, en direct.